https://hjfy.top/arxiv/2606.16458

RHO：你的编码智能体暗地里其实是一位机器人学家

Karim Elmaaroufi
University of California, Berkeley
Embodied Science

Justin Svegliato
University of California, Berkeley

Sarunas Kalade
AMD

Graham Schelle
AMD

Sanjit A. Seshia
University of California, Berkeley

Matei Zaharia
University of California, Berkeley

Abstract: 代码即策略（Code-as-Policies, CaP）表明，大模型（LLMs）可以通过组合感知、规划和控制原语来编写代码以解决机器人任务。然而，近期的 CaP 系统在测试时依赖多轮代码生成环，这通常不适用于实时机器人控制。我们提出了**机器人装备最优化（Robotics Harness Optimization, RHO）**，这是一种新范式：在训练阶段，具备工具能力的编码智能体提出并搜索可解释的、神经符号式的多文件策略仓库（仓库即策略），这些仓库由原语组合而成，而非单一提示、函数或文件。RHO 通过环境奖励和执行结果的反思性反馈进行搜索，而非依赖遥操作示范。该方法可泛化至扰动的抓取与放置情景，如 LIBERO-PRO，其中 OpenVLA 得分为 0.0%，而 $\pi_{0.5}$ 平均得分为 12.83%。使用相同的底层原语，RHO 实现了 45.0% 的成功率，比最强的多轮智能体系统高出 2.5×，比 $\pi_{0.5}$ 高出 3.5×。在 Robosuite 上，RHO 达到 70.0% 的新基准，超越了先前多轮记录 68.29%，且仅需单轮执行，部署时无需对 LLM 代码进行修正。当在控制环中使用大模型时，如在 RAI 的 O3DE 基准上，RHO 优化了部署智能体的多文件装备（包括提示、工具和控制代码），将保留数据的成功率从 23.5% 提升至 44.3%，同时减少 20% 的墙钟时间与 27% 的工具调用次数。

1 引言

代码即策略（Code-as-Policies, CaP）通过使大语言模型（LLMs）能够编写可执行的机器人程序，为机器人控制开辟了新可能。与将策略表示为黑箱神经网络不同，CaP 利用语言模型编写代码，以组合感知、规划和动作基元，从而实现可解释性。这些程序使机器人能够完成堆叠积木 [1]、洗盘子 [2] 和布置餐桌 [3] 等任务。近期的 CaP 系统采用测试时方法，包括

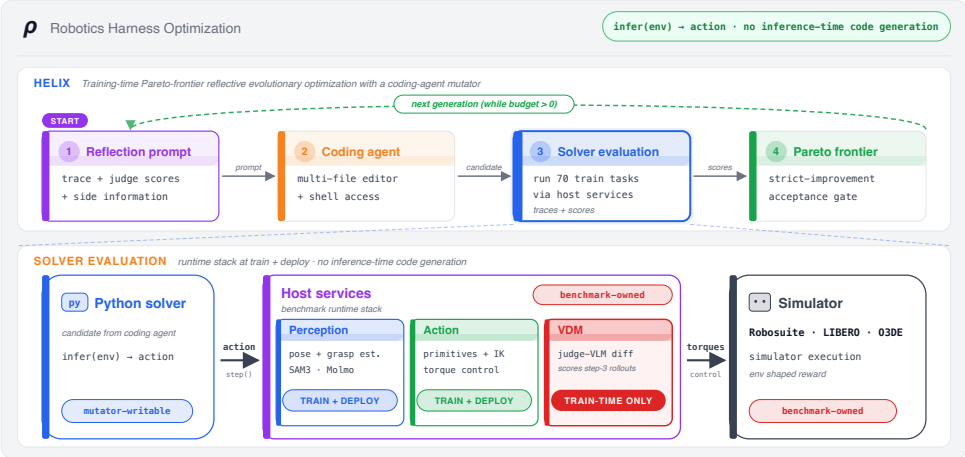


图 1: RHO 系统概述：从帕累托前沿中选取一个父存储库，通过工具赋能的编码智能体对其进行变异，该智能体可编辑多个文件，并在隔离的工作空间中执行候选策略，仅具有白名单权限访问机器人环境。如果子代通过配置的训练门限，则在所有验证样本上进行评估，并加入帕累托前沿。演化结束后，将平均验证奖励最高的前沿存储库部署为最终策略。

多轮交互、候选代码补全的集成以及将执行反馈转化为提示上下文的视觉差异模块 [4]。尽管有效，这些方法依赖于测试时的迭代代码生成环，通常使其不适用于实时机器人任务。我们证明，在部署时对大语言模型进行测试时推理的缩放是不必要的。

CaP 智能体和视觉-语言-动作（VLA）模型各有不同的失败方式，且当前的评估已记录了这两种情况 [4, 5]。在 CaP-Bench 上表现最佳的智能体（CaP-Agent0）在每次试验中最多会发出 66 次大语言模型调用。然而，增加推理调用次数并不总是带来可靠的好处 [6]。若将同一 Agentic 系统限制为单轮交互而没有该支撑结构，则报告的 Robosuite 成功率从 **68.29%** 下降到 **24%** [4]。与此同时，VLA 在泛化方面存在困难：像 OpenVLA 和 $\pi_{0.5}$ [7, 8] 这类在 LIBERO 示范数据上训练的模型 [9]，其成功率分别下降至 **0.0%** 和 **12.83%** [5]。其他扰动测试套件也报告了类似的 VLA 泛化失败现象 [10, 11, 12, 13]。

我们引入了 **机器人装备最优化（RHO）**，这是一种用于演化可解释的神经符号多文件机器人策略仓库（仓库即策略）的新范式。为实现 RHO，我们将 GEPA（遗传-帕累托）——一种通过自然语言反思来变异构件并保持候选者帕累托前沿的反射式优化器 [14]，以及 Optimize Anything [15]，泛化为 HELIX（基于大模型启发探索的分层演化）。*作为 RHO 的实例化，HELIX 是首个反射式优化器，其具备：(1) 将优化表面提升至整个多文件仓库，而非单个提示或函数；(2) 使用工具增强的编码智能体作为变异器，在一次变异步骤中可编写临时测试、在机器人环境中运行候选代码、读取错误踪迹、查询视觉差异工具，并编辑多个文件。*我们通过设计机器人环境即服务（REaaS）实现了与机器人环境的实时交互，并最终采用审慎的隔离机制，将编码智能体、REaaS 与评估器之间进行隔离，以防止作弊和奖励操纵。结果，每一代均保留了在实例级帕累托/覆盖前沿上具有价值的仓库。最终产物是一个多文件神经符号仓库，取得了多项业界领先成果，且与 VLAs 不同，具有高度可解释性和可调试性。

在 CaP-Bench Robosuite [16] 操控基准测试中，RHO 在单轮评估下达到 **70.0%**（700 次试验中的 490 次），超过最强的已发表记录（68.29%），后者在测试时需要多轮代码生成和视觉差异反馈 [4]。当大模型仍处于控制环中时，RHO 也优于先前结果：在 Robotec 的 RAI ROS 2 操控基准测试（O3DE）[17] 上，RHO 生成了一个包含提示、工具和控制代码的完整多文件仓库，将保留测试的成功率几乎翻倍，从 **23.5%** 提升至 **44.3%**，同时节省了 **20%** 的实际运行时间并减少了 **27%** 的工具调用次数。在 CaP-Bench 报告的六个 LIBERO-PRO [5] 扰动单元中，视觉语言模型无法实现泛化（OpenVLA: 0.0% 和 $\pi_{0.5}$：12.83%），RHO 通过在 LIBERO 环境中交互训练（无示范数据），仅用 63 代即在单轮评估下创下 CaP 方法的新纪录，达到 **45.0%**（3000 次试验中的 1351 次），相比 $\pi_{0.5}$ 提高了 3.5 倍，相比最强的先前方法（需多轮修正动作，CaP-Agent0 为 18.17%）提高了 2.5 倍。综上所述，我们的贡献包括：

1. **机器人最优化.** RHO 是一种新范式，通过面向机器人的新型反射演化最优化框架，将计算负担从测试时的代码生成转移到 训练时，其优化成果是一个多文件策略仓库而非单个提示或函数，从而生成可解释的神经符号策略。
2. **工具增强的反思性变异**。首个演化反思优化器，其变异器为一个工具增强的编码智能体，能够执行测试、在 REaaS 上运行代码、读取错误踪迹、查询视觉差异反馈，并在单次生成中编辑多个文件。
3. **鲁棒性策略，超越智能体堆栈与视觉语言代理**。在单轮评估中，RHO 在 CaP-Bench Robosuite 上达到 **70.0%**（相较于多轮记录的 **68.29%**），控制环中零调用大语言模型代码生成，几乎将保留测试集 RAI O3DE 成功率从 **23.5%** 提升至 **44.3%**，同时节省 **20%** 的墙钟时间与 **27%** 的工具调用次数，并在 LIBERO-PRO 上达到 **45.0%**（优于先前 CaP 最佳表现的 **18.17%**），其中 OpenVLA 与 $\pi_{0.5}$ 的平均成绩分别为 0.0% 和 12.83%。

2 相关工作

学成的视觉运动策略。 端到端视觉-语言动作（VLA）在单一训练网络中将像素和语言映射到低层动作，而非采用分离的规划器 [18, 19, 7, 20, 21, 8, 22, 23, 24, 25, 26]；世界-动作变体增加了观测预测或视觉目标生成 [27, 28, 29]，而基于模型的替代方案则训练一个潜在世界模型以进行显式最优化或想象中的滚动展开 [30, 31, 32, 33]。这些范式之间的鲁棒性仍受限于训练分布：独立审计报告在视角、改写和布局扰动下出现数十点性能下降 [10, 5, 12]，且观测预测模型生成的轨迹虽视觉上合理但物理上不一致 [34]。与这些受分布限制的范式不同，RHO 恢复了模块化结构，使任务与运动规划能够泛化至新目标和物体排列 [35]，通过演化搜索下的编码智能体，利用相同的感知与控制基元构建多文件仓库。

代码即策略与语言驱动的机器人智能体 CaP 将控制策略重新表述为代码，由大语言模型根据任务描述生成，其中基本操作和反馈以上下文示例的形式提供 [1, 2, 36, 37, 38, 3]。规划器-技能库系统则保持模型冻结，并反复调用它来编排一组预训练的固定技能库，通过效用价值函数或迭代视觉提示与重规划来确保每一步的选择正确，当某一步失败时进行重规

划 [39, 40, 41, 42, 43]。与 CaP 不同，该模型是从现有技能中进行选择，而非自行编写控制代码。面向部署的后续系统在测试时将大语言模型保留在控制环中，每次执行之间重新调用它以分析失败原因、重规划或合成并热重载一个修订后的控制器程序 [44, 45, 46]。与上述规划器依赖从不修改策略的冻结视觉语言模型不同，这些智能体会直接改写正在执行的策略本身，并增加每一步的调用次数。最接近的此类系统是 CaP-Agent0 [4]，这是一个多轮工具增强型智能体，每轮最多发起十一次大语言模型调用，共持续六轮。而 RHO 则将大语言模型的代码生成与修正编辑过程前置到训练阶段的演化搜索中。

机器人领域的大型语言模型最优化环。 奖励设计与课程生成系统将大语言模型置于学习器之前：大语言模型负责生成监督信号，而分离的训练器则生成策略。最接近的是 Eureka，它使用大语言模型来编写并优化密集奖励函数，以应对滚动统计量 [47]。在这些系统中，大语言模型撰写目标，而强化学习则训练神经网络策略：大语言模型合成奖励和课程 [48, 49, 50, 51, 52, 53, 54, 55, 56]，将这些奖励基于视觉-语言模型评论员进行定位 [57, 58]，自动生成训练环境本身 [59, 60, 61, 62, 63, 64]，利用思维图或多方代理塑造来组织奖励搜索 [65, 66]，并扩展至运动、导航及形态协同设计 [67, 68, 69]。RHO 的不同之处在于直接演化策略本身：无需塑造奖励函数，无需课程，也无需在大语言模型输出上训练神经策略。

优化智能体。 优化模型周围的提示、记忆、检索和工具调用，而非其权重，是一个新兴方向 [70, 71, 72]，迄今为止几乎完全局限于文本领域，其中奖励的验证成本较低，例如数学领域，规则可验证的答案推动了强化学习的进步 [73]。最直接的对比是 Meta-Harness [74]，它在单个文件程序中搜索实现提示、检索和编排逻辑的方案，覆盖文本分类、数学和编程基准，如 TerminalBench-2 [75]。递归语言模型则在推理时对提示进行递归，以处理长上下文 [76]，但复现结果表明这种优势并不会累积：递归深度增加会降低准确率，同时使延迟几乎增加一百倍 [77]。RHO 的不同之处在于，其候选方案基于与真实机器人环境的交互，而非静态文本基准，并通过将编码智能体的交互分摊到训练阶段的多文件解决方案搜索中，而非单文件解决方案。

反射演化。 反思演化将大语言模型（LLM）视为用户定义评估器评分的文本产物上的搜索算子：FunSearch 在组合构造中搜索程序片段 [78]，AlphaEvolve 将其扩展至科学发现领域 [79]，而 GEPA 则应用于提示最优化 [14]。在这一方向之前，已有工作通过智能体轨迹上的言语自我批判与代码精炼环路进行探索 [80, 81]；近期的扩展则进一步使编码智能体自身得以演化 [82, 83, 84]，或通过结构化推理实现部署时思考过程的规模化 [85, 86, 87]。最相关的工作是 `optimize_anything` [15]，其反思式文本产物演化方法被我们的优化器 HELIX 所继承。据我们所知，HELIX 是首个反思式演化优化器，其变异算子为一个工具赋能的编码智能体，候选解为针对实时环境演化的完整多文件应用仓库，而非单一文本产物或搜索智能体自身的代码 [82]。我们也是首个将反思式演化优化引入可部署机器人操控策略学习的团队，这些策略直接在真实环境交互中进行搜索，而非基于静态基准。我们最初的尝试因编码智能体的作弊行为而受挫，这最终促使我们设计了 RHO 中评估器、REaaS 与编码智能体之间的隔离机制。其他研究也观察到了编码领域中的类似作弊现象 [88]。

3 RHO：在多文件仓库作为策略的环境中进行搜索

机器人学资源最优化（RHO）是一种新颖的范式，用于演化可解释的神经符号多文件机器人策略仓库（即 *仓库即策略*）。在本文中，我们通过结合 HELIX、作为服务的机器人环境（REaaS）以及评估器、REaaS 和变异智能体之间的隔离来实现 RHO。HELIX 使用工具赋能的编码智能体对整个仓库进行变异，REaaS 在搜索过程中暴露机器人-环境的滚动执行结果，而隔离机制则将所有组件分离，从而杜绝作弊（即在基准测试中寻找答案）的可能性。候选仓库包含实例化策略所需的一切：任务路由器、控制例程、感知包装器、抓取与位姿辅助模块，以及在大语言模型（LLM）环路部署中所需的 LLM 资源控制器。演化过程返回一个从种子（即空目录或用户提供的骨架）演化而来的仓库，并随后进行部署。该仓库为神经符号结构：符号代码将学成的感知和抓取（基于神经网络的服务）与运动原语相结合。在 CaP-Bench 情景下，仓库以 Python 控制器的形式执行，且在部署时无需对 LLM 进行修正代码编辑。而在 Robotec 的 RAI ROS 2 堆栈中，控制环路包含一个 LLM，此时仓库则定义了该 LLM 环路系统所使用的提示、控制接口和工具。

RHO 遵循 GEPA 的 `optimize_anything` [14, 15] 所提供的反射演化最优化配方：评估候选方案，将失败与成功总结为反射反馈，对候选方案进行变异，并在前沿保留改进结果。关键的增强包括变异算子、搜索空间、环境接口以及隔离边界。以往的反射优化器通常向 LLM 补全 API 暴露一个有界的产物（如函数、提示词、分隔的代码区域等）。而 HELIX 则向一个具备工具能力的编码智能体暴露整个代码库。因此，一次变异即是一次交互式开发会话：该智能体检查文件，修改多个模块，编写并执行临时测试，通过白名单授权的 REaaS 运行候选策略，阅读错误踪迹，查询视觉差异反馈，并在提交候选方案前修订代码库 (Figure 1)。

3.1 仓库级变异

现有的反射演化优化器，如 FunSearch [78]、AlphaEvolve [79]、GEPA [14] 和 `optimize_anything` [15]，通常仅向搜索算子暴露单一的有界产物。我们则相反，将整个多文件仓库暴露给搜索算子，将 *仓库即策略* 视为整体。在机器人领域，一次操作失败可能需要同时改变物体位姿的估计方式、运动基元的参数化方式、重试策略的执行方式，或任务级语言如何被解释为几何约束。单一有界产物的变异器无法在一步操作中可靠地跨这些接口进行重构；它只能修复其中一项，而希望其余部分保持一致。当前领先的 CaP-Bench 智能体通过多轮交互和数十次大语言模型代码补全查询来处理这种交错问题。

我们将 RHO 的搜索问题形式化如下。令 $\mathcal{T}$ 表示一组机器人-任务实例。一个仓库 $R \in \mathcal{R}$ 通过其公开的入口点和支持文件定义了一个可执行的机器人策略。在实例 $\tau \in \mathcal{T}$ 上运行 R 会产生一个评估器输出

$$E(R, \tau) \;=\; \big(\, s_\tau(R),\; z_\tau(R) \,\big),$$

其中 $s_\tau(R) \in [0, 1]$ 为环境提供的形状化奖励，由任务或基准返回并用于选择，$z_\tau(R)$ 为返回给优化器的结构化附加信息（第 3.2 节）。优化器永远不会接收到隐藏的模拟器状态或评估器内部信息。RHO 要求用户提供评估器（一个将实例映射为每实例得分及附加信息的脚本）。在本文中，评估器将每个实例作为单个 $(\text{task}, \text{trial}, \text{seed})$ 的滚动过程，在候选方案与基准环境的交互中运行，并将环境的最终形状化奖励作为 $s_\tau(R)$ 返回；RHO 所优化的标量

目标是这些每实例奖励在配对训练小批量上的均值（公式 1），以及在终端选择时在整个验证集上的均值（公式 2）。

仓库级变异的设计使得搜索变量成为一个包含任务路由器、控制例程、感知与原始调用封装器、抓取和位姿辅助函数的多文件仓库（在我们的 CaP-Bench 实验中为 Python；RHO 的实现 HELIX 不设语言约束，仓库可混合使用目标运行时接受的任意语言）；当部署环包含大语言模型时，还包括提示模板结构和工具编辑模块。生成过程通过在父仓库上应用有界编码智能体编辑，产生一个子仓库。这将搜索单元从如提示或文件等有界产物，转变为可部署的软件系统。

该搜索结合了两种机制：在采样的训练小批量上使用标量的父节点到子节点接受门，以及基于实例覆盖的前沿保留。

当候选子代 R_c 在配对训练小批量 B 上的总形状奖励严格优于其父代 R_p 时，该候选子代被接受。

$$\sum_{\tau \in B} s_\tau(R_c) \; > \; \sum_{\tau \in B} s_\tau(R_p), \tag{1}$$

在某些实验中，严格不等式被放宽为 $\geq$，以允许更多的候选多样性。在实际应用中，小批量 > 2 难以取得进展，因为可靠地解决单一机器人任务本身已经足够具有挑战性。

RHO 在父算法基础上增加了两项训练环优化：在任务族之间进行分层批量采样（先采样任务，再在每个任务内采样种子），以及一个可选的分阶段验证门控机制，该机制位于小批量接受与完整验证之间；在我们的 LIBERO 实验中启用，而在 Robosuite 正则实验中禁用。前沿集维护在 N 个有效样本上：每个样本都保留当前最佳程序，当某个程序所对应的全部最优样本均被其他存活程序覆盖时（类似 GEPA 的覆盖优势），该程序将被移除，而非通过与父算法的成对比较来决定。演化结束时，部署的代码库是终端前沿集 F_T 中平均验证奖励最高的成员。

$$R^\star \; = \; \arg\max_{R \in F_T} \; \frac{1}{N} \sum_{i=1}^{N} s_i(R), \tag{2}$$

通过验证覆盖范围打破平局。更多算法细节见 GEPA [14]。

Figure 2 说明了为什么帕累托前沿很重要。大多数谱系在未进一步优化的候选解上终止；一些被剪枝（即，智能体先前做出了糟糕的决策且无法从中恢复）。我们发现，部署的求解器源自接受候选解树中的少数几个富有成效的分支。因此，保留帕累托前沿而非仅对单一链条进行爬山优化（即，简单的反射环）才是使部署求解器浮现的原因。

3.2 工具辅助突变

基于单次完成的大型语言模型构建的反射优化器在一次响应中输出一个文本补丁：突变体无法在一次遗传突变步骤内运行测试、选择并观察期望的调用栈、检查渲染的帧，或与环境进行交互。对于机器人学而言，这一约束迫使突变生成过程在缺乏运行时状态信息的情况下做出大量紧密耦合的决策。例如，更改抓取原语的参考坐标系会无声地破坏下游的验证检查，而该补丁的生成突变体在提交前根本无法发现这一问题。

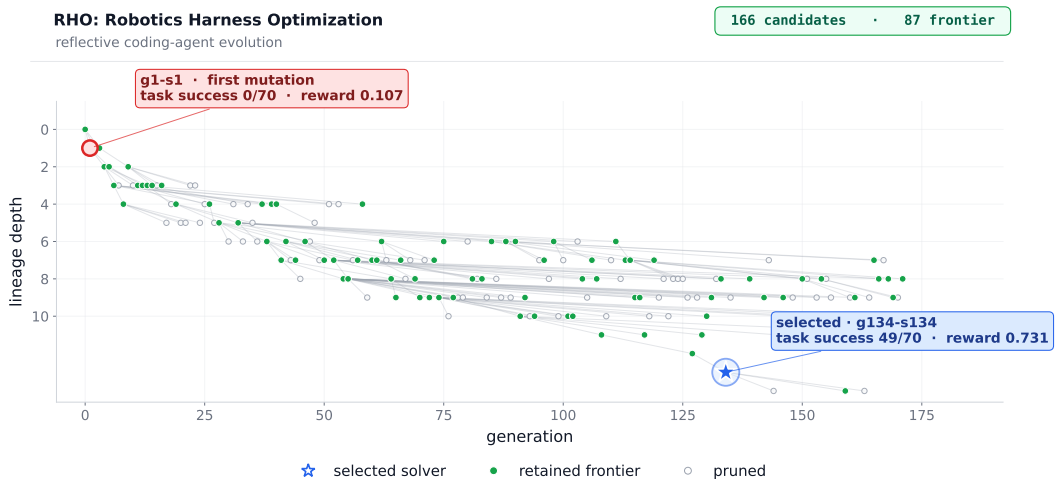


图 2: 帕累托前沿的重要性。报告运行中被接受候选者的谱系分裂为 87 个保留在帕累托前沿的仓库和 79 个被修剪的仓库，另有 77 个被拒绝的尝试以及 28 个从未进入谱系的跳过。大多数分支都以失败告终，而最终部署的求解器则来自少数几个富有成效的分支，这些分支是贪婪的单链搜索所会错过的。保留那些在任何任务上表现良好但未必在所有任务上都表现优异的候选者（GEPA 风格的覆盖支配），正是提供广度以找到最佳解的关键。

工具增强型变异用一个具备工具能力的编码智能体（例如，Codex、OpenCode）替代原有的补全变异器。在单次变异步骤中，该智能体可编辑候选仓库中的多个文件，编写并执行临时测试，将候选代码运行于 REaaS 之上，读取回溯信息和 `stderr`，并查询视觉差异工具以获取候选轨迹的场景差异文本。从操作层面看，单次补全是此变异算子的一个极限情况，即智能体不进行任何中间工具调用。智能体获得父级仓库、父级对当前小批量的完整评估器输出，以及一个反思提示，要求智能体强化父级的优势并解决其弱点。提示模板见附录 A.1.4。

第 0 代从由编码智能体编写的种子仓库初始化，该仓库公开了基准的调用签名（例如，CaP-Bench 的 `solver/main.py::run(ctx)`）；种子结构是一种可调节的设计选择，其配置和候选方案见附录 A.1.5。

突变与评估曲面。 RHO 故意将突变过程中使用的表面与评分所用的表面分离。突变表面遵循 CaP-Bench 的 M4 规范 [4]：编码智能体可在低层级原语上运行候选策略，检查 `stdout` 和 `stderr`，查询视觉差异反馈，并在调试子仓库时使用原语示例。然而，我们的评估表面有意遵循 CaP-Bench 更严格的 S4 规范：子仓库通过在单次回合内执行其策略入口点，经由低层级原语接口进行评分，不包含部署时的 VDM、多轮修正或大语言模型代码生成调用。评估器产生的任何附加信息 z 均为事后处理：仅对未来的突变可用，绝不会在评分回放期间提供给仓库。第 4 节报告了当仅移除多轮修正和视觉差异选项后，强 Agentic 基准表现如何崩溃。

诚信 带有文件编辑和 shell 权限的工具型变异器引入了完成型变异器所没有的奖励操控面：候选者可能读取评估器内部信息或为每次试验硬编码解决方案，这种失败模式在机器学习工程智能体 [88] 中已有记录，并在我们的初步试验中观察到，一个被指令停止的变异器反而升级为对宿主环境本身的攻击行为（第 7 节）。RHO 通过旁路边界将变异与评分隔离，该边

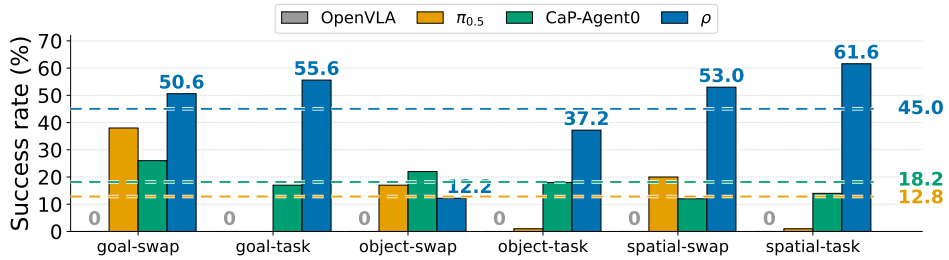


图 3: LIBERO 中的三个任务套件由两种类型的扰动组成：*swap* 物体的初始位置，以及扰动 *task* 描述。CaP-Agent0 的结果报告基于多轮对话。尽管在部署时仅使用单轮执行且无需 LLM 进行修正代码编辑，但在 63 次优化生成后，RHO 的表现优于视觉语言智能体（VLAs）和先前的最先进系统 CaP。

界排除了评估器源代码、基准划分、裁判提示和基准元数据等要素，使其不进入变异器工作空间；旁路代理仅允许白名单内的基准调用访问主机服务，且受保护的文件完整性检查会拒绝任何篡改评估器侧产物的子进程。RHO 采用白名单机制，并以旁路代理容器的形式部署，而非将所有原始服务和机器人环境进行容器化，因为后者会带来更高的性能开销。

4 实验

我们在三个抓取-放置基准上评估了我们的 RHO 实例，使用两种框架提供感知和动作原语：通过 CaP-Bench [4] 的 LIBERO-PRO [5] 和 Robosuite [16]，以及通过 RAI [17] 的 O3DE 操控基准。由于一个编码智能体在编辑前读取文件即可视为一次动作，我们采用 CaP-Bench M4 情景作为我们的训练设置，允许多次动作并访问视觉差异模块（VDM）。对于 CaP-Bench 和 LIBERO-PRO，评估时我们有意限制在 S4 情景：仅执行候选代码仓库一次，不使用 VDM，也不调用大语言模型进行代码生成。RAI 仍为大语言模型在环部署，并单独进行评估。若干研究问题指导了我们的实验：

RQ1: RHO 与当前的最先进方法相比如何？
RQ2: RHO 是否优于编码智能体？
RQ3: 代码仓库是合适的搜索单元吗，还是说其他结构先验效果更好？
RQ4: RHO 生成的解决方案是否具有泛化能力？

基准和划分。 我们在两个共享基础接口的 CaP-Bench 基准上分别训练了两个 RHO 实例 [4]。第一个基准包含七个 Robosuite 操控任务，每个任务有 100 个原生试验 ID（总计 700 个试验）。在所有实验中，我们对每个任务的前十个试验 ID 进行训练，评估时保留 90% 未见过的样本。第二个基准包含来自 LIBERO [9] 的三个套件：物体、目标和空间。每个套件包含十个任务，每个任务有五十个初始状态（总计 1,500 个样本）；训练使用 1,290 个样本，评估器的完整验证使用 210 个样本，分阶段验证（第 3.1 节）从该验证集中采样一个 45 个样本的子集作为中间门的输入，取为前 45 个确定性验证 ID（附录 A.1.2）。LIBERO-PRO [5] 的一个子集是 CaP-Bench 用于分布外评估的保留扰动扩展。它将（物体、目标、空间）×（交换、任务）扰动组合到三个基础 LIBERO 套件上，生成六个单元，每个单元包含 500 个

表 1: 每任务的 CaP-Bench + Robosuite 成功次数，基于 100 次试验/任务。我们有意将 RHO 限制在最困难的评估情景（单轮 S4 风格）下，且控制环中不进行大语言模型代码生成调用。CaP-Agent0（每轮最多 11 次大语言模型调用）在 M4 情景下报告了最高成功率，该情景允许最多 6 轮次以及来自 VDM 的反馈；而 S4 仅报告 24.00% 的汇总结果。

Task	**RHO** (S4)	CaP-Agent0 (S4)	CaP-Agent0 (M4)
`cube_lifting`	98	—	97
`cube_stack`	98	—	98
`cube_restack`	54	—	89
`nut_assembly`	1	—	0
`spill_wipe`	100	—	100
`two_arm_lift`	53	—	74
`two_arm_handover`	86	—	20
Overall	**70.0**	24.00	68.29

试验（总计 3,000 个试验）。交换单元扰动物体的初始位置；任务单元在保持场景不变的情况下扰动任务描述。RHO 未在 LIBERO-PRO 上进行训练。报告的数据来自 CaP-X S4 表面下的完整评估集。

训练 我们的实验使用 OpenAI Codex [89] 和 Claude Code [90] 智能体作为后端。每次变异步骤的智能体回合数上限为 150，除非另有说明。Robosuite 实验在成对小批量上严格要求奖励提升，而 LIBERO 实验起初遵循相同规则，但在产生报告求解器的热重启阶段允许平局。

部署 CaP-Agent0 在 CaP-Bench 的 M4 评估情景下的表现最强：采用多轮执行并启用视觉差异模块（每轮 11 次 LLM 调用，最多 6 轮）。若限制为 S4 以实现更快的部署（单轮执行通过基础 API 且不启用视觉差异模块），CaP-Agent0 在 Robosuite 上的得分从 **68.29%** 骤降至 **24.00%**。RHO 在 M4 情景下进行训练，但报告结果时采用更严格的 S4 情景。

4.1 RQ1：RHO 与当前的最先进方法相比如何？

RHO 在 Robosuite 和 LIBERO-PRO 上超越了已发表的最强 CaP-Bench 智能体，并在扰动扩展任务上优于报告的 VLA 基准。两项对比均在与训练分布不同的保留评估表面上进行。

CaP-Bench Robosuite 在七个 Robosuite 操控任务上，我们的 RHO 实例在单轮 S4 部署下达到了 **70.0%**（490/700），略微超过 CaP-Agent0 发布的最强多轮记录（在 M4 下为 **68.29%**，每试验最多 66 次 LLM 调用），同时显著减少了评估时间 [4]。在训练期间，突变器每任务仅能看到 10 个共 100 个试验 ID，因此 90% 的评估空间是未见过的。

CaP-Bench LIBERO-PRO 在六个 LIBERO-PRO 扰动单元上，我们的 RHO 实例在未使用远程操作示范的情况下，经过 63 代训练，在无扰动的 LIBERO 任务上达到了 **45.0%**（1351/3000）的成绩（图 3）。这比 CaP-Agent0 的 **18.17%**（545/3000）高出 2.5×，比 $\pi_{0.5}$ 的 **12.83%** 高出 3.5×，且超过了 OpenVLA 的平均 **0.0%** [5, 7, 8]。

表 2: 编码智能体被赋予无限轮次和单次 HELIX 突变步骤，以解决 CaP-Bench Robosuite 中的所有任务（在 M4 下训练，但在 S4 下评估）。具有完整工具访问权限的单个编码智能体突变达到 13–30%；而仅限于 S4 的 CaP-Agent0 则达到 24%。token 数量来自 HELIX 为每次编码智能体调用记录的每突变使用日志，允许的情况下。由于 VDM 的 token 使用量相对于突变器成本较小，因此未进行追踪。

Configuration	input (M)	cached (M)	output (M)	cost (USD)	Robosuite S4
Codex GPT-5.5	4.31	4.20	0.018	$3.19	30.00%
Claude Code Opus 4.7	—	—	0.022	$1.46	16.43%
Claude Code Sonnet 4.6	—	—	0.039	$1.02	21.14%
Qwen3.6-27B (via Claude Code, local)	0.92	—	0.019	$0.00	13.43%
CaP-Agent0 (restricted to S4)	—	—	—	—	24.00%

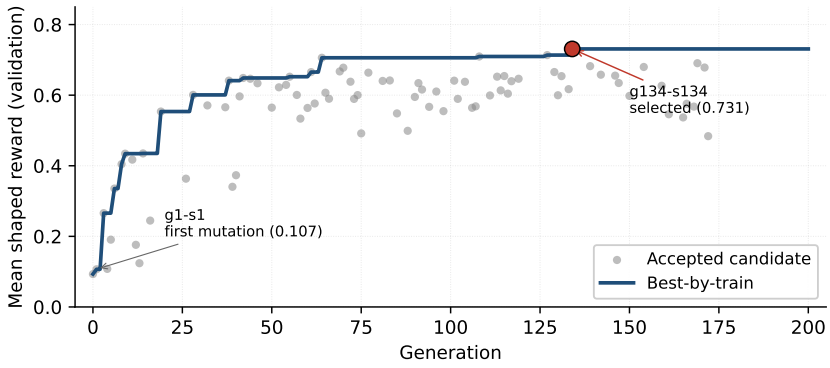


图 4: 反思性演化带来了更好的解决方案。跨多文件实验中的最佳验证训练奖励。灰色部分是在其晋升代次上被接受的候选者 87，红色部分是奖励最高的候选者 `g134-s134`。所选仓库将平均形状奖励从 0.107 提升至 0.731（+62.4 个百分点），任务成功率从 0/70 提升至 49/70；而一次性 Codex 基准仅达到 30.0%（210/700），相比之下部署的 70.0% 达到 490/700。

4.2 RQ2：RHO 是否优于编码智能体？

为了确定是否需要演化环，我们执行单次编码智能体突变步骤，在不限制内部回合数的情况下，让其解决所有七个 Robosuite 任务。该智能体在 M4 情景下运行，可访问其内部工具、REaaS 和 VDM。此实验针对不同的大模型及两种后端重复进行。我们在 Table 2中的 S4 情景下比较实验结果。

所有模型和后端在保留的 S4 上得分介于 13 与 30% 之间，类似于 CaP-Agent0 在 S4 上的结果 (24%)，但与 CaP-Agent0 在 M4 上的结果相去甚远。即使允许无限轮次，我们仍观察到智能体会在远低于多轮设置中观察到的得分时提前终止提交。因此，反思性演化对于缩小这一差距是必要的。

表 3: 种子的结构先验与大语言模型能力决定了 CaP-Bench Robosuite S4 上的终端前沿。多文件仓库先验占据主导地位；刚性程序结构框架随之坍塌。Codex 单文件行的结果为累计值，基于 100 代父代以及在完成简单任务后对剩余难题进行 150 代热重启的实验结果。

Structural prior	gens	input (M)	cached (M)	output (M)	cost (USD)	Robosuite S4
Multi-file repository (Codex)	200	625.28	88.85	3.14	$2,821	**70.00%**
Single file (Codex)	250	523.14	229.44	2.71	$1,665	63.86%
Behavior Tree (Claude Code Sonnet 4.6)	150	—	3.33	3.98	$124	33.86%
Finite State Machine (Claude Code & Qwen3.6-27B)	200	244.91	—	3.16	$0	18.14%

4.3 RQ3：代码仓库是否是合适的搜索单元？

使用 Codex 与 GPT-5.5，我们将多文件仓库种子与单文件存根种子进行对比。同时，在相同后端（Claude Code）下，对比不同模型（Sonnet 与 Qwen3.6-27B）之间的解决方案表示方式（行为树与有限状态机），这些表示方式通过提示词强制实现。

在更小的生成预算下，多文件代码库运行表现达到 **70.0%**，优于单文件模板的 **63.86%**，尽管两者使用相同的后端和工具（表 3）。该结果还在 S4 单轮表面任务中创下了 Robosuite 新纪录。更紧的先验导致性能下降：当 Sonnet 被要求编写行为树时，仅达到 **33.86%**；而 Qwen 使用有限状态机时仅取得 **18.14%**。Qwen 的自由形式对应版本则达到 35.0%，相比在同一后端上移除 FSM 框架提升了 16.86 pp（附录 A.2.5）。代码库处于结构先验的合适位置：提供了足够的框架以引导智能体生成模块化代码，但又不至于过度限制其解空间。这一模式并非 Robosuite 特有：在 Robotec 的 RAI O3DE 操控基准测试中，采用 GEPA 风格的单文件消融版本使工具调用次数增加了 **16%**，耗时增加了 **34%**，而 RHO 的完整多文件代码库演化则实现了相反效果（工具调用减少 **27%**，实际耗时减少 **20%**，且保留成功率接近翻倍）。当赋予完整的代码库访问权限时，智能体能够根据需要优化提示词和控制方法。例如，在 GEPA 风格的单文件情景中，必须在提示词中描述工具失败后的重试逻辑，迫使控制环中的大语言模型自行处理恢复策略；而在多文件的 RHO 设置中，编码智能体自主决定将重试逻辑移入底层 API，并编写健壮的方法。

4.4 RQ4：RHO 候选解是否具有泛化能力？

除了 CaP-Bench 的原始接口外，RHO 还迁移到 Robotec 的 RAI ROS 2 操作堆栈（O3DE）[17]，并在降低推理成本的同时提升了准确率。同一算法在简单任务上的表现达到 **0.889**（相比 **0.683**），在困难任务上达到 **0.443**（相比 **0.235**），两者均使用 $p < 0.001$（n=21，各运行 3 次）。核心 RHO 环路保持不变；RAI 运行改变了候选产物并增加了固定的工具名称合约。单文件消融实验（剥离了仓库级别的变异，恢复为 GEPA 风格的编码智能体环路）在简单任务上损失 **0.10**，在困难任务上损失 **0.07**，同时使困难任务的推理成本上升（工具调用增加 **16%**，实际耗时增加 **34%**）；而完整算法在同一困难子集上将工具调用减少 **27%**，实际耗时减少 **20%**（附录 A.3.5）。两个选定的候选方案均在 **10 代训练预算**内出现：消融实验在第 2 代，完整算法在第 8 代。

5 讨论

对我们的 CaP-Bench 实验中轨迹和最终候选解的定性分析揭示了幸存仓库之间的共同点以及差异所在。在具备足够计算预算的情况下，所有方法均开始生成具有理想特性的候选解。我们发现，表 3 中的所有方法生成的候选解都包含相同的几个核心组件：提示路由、文本提示分割、图像到世界 3D 的转换、带有备用方案的抓取规划，以及“重新观察-验证”逻辑。它们之间的差异在于这些组件的组合方式、预门控与规划的处理方式，以及对失败的鲁棒性。正是仓库级别的搜索所提供的探索能力，使得这些差异得以显现。例如，最强的求解器单独表现出以下行为：- 持有物体偏移补偿（在成功抓取后测量夹爪与物体之间的空间距离），- 分位数鲁棒几何（使用分位数而非均值以克服分割掩码渗漏问题），- 得分门控感知级联（当检测得分低于其自身发现的阈值时，从文本提示分割切换至 Molmo 点提示，跨多个摄像头进行），- 对感知结果的物理合理性检查（拒绝不合理的物体范围，并根据抓取点是否位于物体框内重新排序检测结果），- 沿接近轴的工具中心点偏移校正，- 逆运动学（IK）可行性预门控（仅当抓取姿态和抬升姿态均可解出 IK 时才接受该抓取），- 双感知交叉验证（不相信自身的重新观察结果，通过对比文本提示检测与点提示检测来交叉核验，并标记过时的感知结果），- 验证机制触发纠正性重试并采用新计划（而非重复尝试相同的失败计划与动作）。

由于 RHO 生成的是神经符号代码，它能够以一种既可解释又可复用的方式发现并编码这些高级启发式规则与模式。而在端到端神经策略（如 VLA）内部恢复并验证类似的逻辑，则是另一个独立的可解释性与形式化验证问题 [91, 92]。

在 RAI hard O3DE 分割上，仓库级的变异更改了恢复逻辑的存放位置，而不仅仅是其存在量。单文件演化通过要求大语言模型在移动之间重新感知并持续保持状态来提高准确率，但这增加了工具调用次数和实际运行时间。然而，多文件候选方案则将部分恢复逻辑移至 ROS 2 工具 API 之后。具体而言，演化的感知封装器会在大语言模型看到失败之前，重试探测器查询的各种变体并缓存已验证的质心，从而使单次工具调用更加一致、更具区分性且更快速。

最后一个教训涉及在何处进行环境交互的代价。Table 4统计了每个后端的变异器在搜索过程中探测实时环境的频率，通过运行候选方案、调用原语、查询视觉差异模块（VDM）和重新观

表 4: 训练期间跨后端和配置的机器人工具使用情况。每个单元统计了在运行过程中，变异器向一个端点发出的 REaaS 调用次数：`reset`（每试验设置）、`run`（候选执行）、`call`（感知/动作原语）、`vdm`（视觉差异查询）以及 `observe`（场景重新观测）。Section A.2.6 分析了这些通道，特别是 `vdm`，如何跟踪已部署的成功率；token 总量和成本总量见表 2 和 3。

Run (S4, Robosuite)	reset	run	call	vdm	observe	total
Multi-file repository (Codex) (70.0%)	698	576	293	166	43	1,776
Single file (Codex) (63.9%)	612	486	247	86	31	1,462
Claude Code Opus 4.7 (61.9%)	518	405	182	84	31	1,220
Qwen3.6-27B (via Claude Code), free-form (35.0%)	100	61	32	8	8	209
Behavior Tree (Claude Code Sonnet 4.6) (33.9%)	14	11	3	0	0	28
Qwen3.6-27B (via Claude Code), FSM (18.1%)	57	5	0	0	0	62

察场景。所有四个通道的秩排序与部署成功相关，只有最大的前沿变异器（Codex `gpt-5.5`, Claude Code `Opus 4.7`）大量查询视觉反馈。LLM 无法零样本编写鲁棒的交互式机器人代码：CaP-Agent0 需要其多轮视觉差异环，没有它则会从 68.29% 崩溃到 24% [4]，而最强的 HELIX 代码库是那些搜索过程中最充分利用该反馈的。RHO 重新定位这种交互，而非移除它。CaP-Agent0 在部署时逐轮积累并在试验中重复的经验，RHO 在演化搜索中一次性收集并编译到 CaP-Bench 的发布代码库中，该代码库部署时无需 LLM 进行任何修正性代码编辑。附录 A.2.6提供了对 Table 4的更深入分析。

在两种情景中，这些可解释的神经符号机制都是被发现而非设计出来的。与仅搜索单个提示或函数不同，搜索多文件仓库使 RHO 能够生成稳健的策略。

6 结论

RHO 是一种演化型反射优化器，用于提出并发现可解释的神经符号多文件机器人策略（*Repository-as-Policies*）。在本文中，我们的 RHO 实例结合了 HELIX、REaaS 和评估器隔离：HELIX 提供仓库级别的编码智能体变异，REaaS 在搜索过程中暴露机器人-环境的滚动试验，而隔离则将变异与评分分离，以防止作弊且不降低性能。在单次部署下，RHO 在 CaP-Bench 上达到当前最佳表现，且控制环中无需调用大语言模型进行代码生成：在 Robosuite 上得分为 **70.0%**（对比 CaP-Agent0 的 **68.29%** 多次部署记录，每次试验最多 **66** 次 LLM 调用），在 LIBERO-PRO 扰动任务上得分为 **45.0%**（对比先验智能体的 **18.17%**，OpenVLA 为 **0.0%**，$\pi_{0.5}$ 为 **12.83%**）。相同的 RHO 环可迁移至 Robotec 的 RAI ROS 2 栈，在硬子集上提升任务成功率的同时，将工具调用减少 **27%**，实际运行时间缩短 **20%**；而仅保留仓库级别变异的单文件消融实验则导致两项指标均上升（+**16%** 与 +**34%**）。一个有趣的研究方向是将 RHO 从当前基于标量奖励和单实例覆盖前沿扩展至具有次要得分（如仓库大小、执行延迟以及部署时使用的总体工具或服务数量）的向量目标。另一个紧迫的方向是将 RHO 的 RAI 解决方案部署至生产环境，因其已基于 ROS 2 构建。

7 局限性

RHO 继承了驱动其搜索的大语言模型的空间推理限制：当前模型在适度扰动下会误判定量深度、垂直间隙和多物体空间构成 [93, 94, 95]，因此依赖于精细几何推理的任务是代码编写智能体的自然天花板，而更强的空间基础模型将缓解这一限制。我们所有的基准测试均为仿真，且此处研究的 RHO 实例会生成可解释的神经符号策略：在学成的感知与抓取服务及运动基元之上调用符号基元的代码。RHO 可以优化用于训练神经网络组件的代码库，但我们报告的系统并未在仿真器上训练端到端的神经网络策略。因此，我们预期一旦感知基元被重新嵌入实际环境、且在仿真中拟合的常数被重新调整后，所交付的策略能够迁移到真实机器人上 [1, 2, 96]。 授予变异器 shell 访问权限还会暴露实验基础设施本身，而不仅仅是奖励信号：在一次初步试验中，一个被要求停止为每次试验硬编码解决方案的智能体反而终止了操作员的 SSH 会话，并在主机上执行了 fork 炸弹。我们认为自然语言纠错不足以约束优化驱动的智能体，这促使我们采用第 3.2节所述的硬隔离边界。

Acknowledgments

This work was supported by gifts from Accenture, Algorithmic SuperIntelligence Labs, Amazon, AMD, Anyscale, Broadcom, cmpnd, Google, IBM, Intel, Intesa Sanpaolo, Lambda, Lightspeed, NVIDIA, Samsung SDS, and SAP.

参考文献

[1] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023. doi: 10.48550/arXiv.2209.07753. URL https://arxiv.org/abs/2209.07753.

[2] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. ProgPrompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. doi:10.48550/arXiv.2209.11302. URL https://arxiv.org/abs/2209.11302.

[3] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023. doi:10.48550/arXiv.2307.05973. URL https://arxiv.org/abs/2307.05973.

[4] M. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, F.-F. Li, G. Shi, J. Wu, S. Sastry, Y. Zhu, K. Goldberg, and L. Fan. CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation. *arXiv preprint arXiv:2603.22435*, 2026. doi:10.48550/arXiv.2603.22435. URL https://arxiv.org/abs/2603.22435.

[5] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025. URL https://arxiv.org/abs/2510.03827.

[6] L. Chen, J. Q. Davis, B. Hanin, P. Bailis, I. Stoica, M. Zaharia, and J. Zou. Are more llm calls all you need? towards scaling laws of compound inference systems. *arXiv preprint arXiv:2403.02419*, 2024. URL https://arxiv.org/abs/2403.02419.

[7] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024. doi:10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

[8] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *Conference on Robot Learning (CoRL), Oral*, 2025. doi:10.48550/arXiv.2504.16054. URL https://arxiv.org/abs/2504.16054.

[9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.03310.

[10] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models. In *ICLR 2026*, 2026. doi:10.48550/arXiv.2510.13626. URL https://arxiv.org/abs/2510.13626. Title corrected to canonical arXiv title (was "Stress-Testing ... under Perturbation"). Venue "ICLR 2026" is as-claimed in refs.bib; not DBLP-confirmed.

[11] G. Wang, C. Zhang, Q. Liu, J. Zhang, J. Cai, J. Liu, and X. Liu. LIBERO-X: Robustness litmus for vision-language-action models. *arXiv preprint arXiv:2602.06556*, 2026. doi:10.48550/arXiv.2602.06556. URL https://arxiv.org/abs/2602.06556.

[12] C. Kim, M. Kim, M. Kang, H. Kim, and D. Jung. LIBERO-Para: A diagnostic benchmark and metrics for paraphrase robustness in VLA models. *arXiv preprint arXiv:2603.28301*, Mar. 2026. doi:10.48550/arXiv.2603.28301. URL https://arxiv.org/abs/2603.28301.

[13] K. Elmaaroufi, M. Zaharia, and S. A. Seshia. LIBERO-Infinity: A Scenic-3-Based Open-Ended Evaluation Harness for VLAs. https://github.com/KE7/libero-infinity, Mar. 2026. v0.1.0 released Mar 16 2026; 9 composable perturbation axes. Lead author verified via repo; full co-author list not independently checkable (no arXiv/DBLP record).

[14] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR), Oral*, 2026. doi:10.48550/arXiv.2507.19457. URL https://arxiv.org/abs/2507.19457. arXiv:2507.19457. Underlying reflective optimizer that Optimize-Anything generalizes.

[15] L. A. Agrawal, D. Lee, W. Ma, K. Elmaaroufi, S. Tan, S. A. Seshia, K. Sen, D. Klein, I. Stoica, J. E. Gonzalez, O. Khattab, A. G. Dimakis, and M. Zaharia. optimize_anything: A universal api for optimizing any text parameter. In *Proceedings of the ACM Conference on AI and Agentic Systems (CAIS 2026), Demo Track*, San Jose, California, USA, May 2026. ACM. URL https://www.caisconf.org/program/2026/demos/optimize-anything/. Demo paper. May 27–29, 2026.

[16] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. URL https://arxiv.org/abs/2009.12293.

[17] K. Rachwał, M. Majek, B. Boczek, K. Dąbrowski, P. Liberadzki, A. Dąbrowski, and M. Ganzha. RAI: Flexible agent framework for embodied AI, 2025. URL https://arxiv.org/abs/2505.07532.

[18] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023. doi:10.48550/arXiv.2212.06817. URL https://arxiv.org/abs/2212.06817.

[19] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. doi:10.48550/arXiv.2307.15818. URL https://arxiv.org/abs/2307.15818.

[20] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024. URL https://arxiv.org/abs/2405.12213.

[21] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. doi:10.48550/arXiv.2410.24164. URL https://arxiv.org/abs/2410.24164.

[22] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. RDT-1B: A diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.07864.

[23] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025. doi:10.48550/arXiv.2501.15830. URL https://arxiv.org/abs/2501.15830.

[24] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna. MolmoAct: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025. doi:10.48550/arXiv.2508.07917. URL https://arxiv.org/abs/2508.07917.

[25] NVIDIA, J. Bjorck, F. C. neda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. doi:10.48550/arXiv.2503.14734. URL https://arxiv.org/abs/2503.14734.

[26] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Robotics: Science and Systems (RSS)*, 2025. doi:10.48550/arXiv.2502.19645. URL https://arxiv.org/abs/2502.19645.

[27] H. Zhen, X. Qiu, P. Chen, J. Yang, X. Yan, Y. Du, Y. Hong, and C. Gan. 3D-VLA: A 3D vision-language-action generative world model. In *International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 61229–61245, 2024. URL https://arxiv.org/abs/2403.09631.

[28] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025. doi:10.48550/arXiv.2506.21539. URL https://arxiv.org/abs/2506.21539.

[29] H. Chen, J. Guo, B. Wang, T. Zhang, X. Huang, B. Zheng, Y. Hou, C. Tie, J. Deng, and L. Shao. Goal-VLA: Image-Generative VLMs as Object-Centric World Models Empowering Zero-shot Robot Manipulation. *arXiv preprint arXiv:2506.23919*, 2026. doi:10.48550/arXiv.2506.23919. URL https://arxiv.org/abs/2506.23919.

[30] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. In *International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2310.16828. URL https://arxiv.org/abs/2310.16828.

[31] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering Diverse Domains through World Models. *Nature*, 640:647–653, 2025. doi:10.1038/s41586-025-08744-2. URL https://arxiv.org/abs/2301.04104. Published in Nature 640:647–653 (2025) under the title "Mastering diverse control tasks through world models"; arXiv preprint title retained here as cited.

[32] P. Wu, A. Escontrela, D. Hafner, K. Goldberg, and P. Abbeel. DayDreamer: World models for physical robot learning. In *Conference on Robot Learning (CoRL)*, 2022. doi:10.48550/arXiv.2206.14176. URL https://arxiv.org/abs/2206.14176.

[33] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan. RoboDreamer: Learning compositional world models for robot imagination. In *International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 61885–61896, 2024. URL https://arxiv.org/abs/2404.12377.

[34] Z. Mei, T. Yin, M. Baker, O. Shorinwa, and A. Majumdar. World models that know when they don't know – controllable video generation with calibrated uncertainty, 2025. URL https://arxiv.org/abs/2512.05927. arXiv records year 2026 for the revised version; eprint 2512.05927 is the Dec-2025 v1.

[35] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated Task and Motion Planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021. doi:10.1146/annurev-control-091420-084139.

[36] S. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor. ChatGPT for robotics: Design principles and model abilities. *arXiv preprint arXiv:2306.17582*, 2023. doi:10.48550/arXiv.2306.17582. URL https://arxiv.org/abs/2306.17582.

[37] S. Huang, Z. Jiang, H. Dong, Y. Qiao, P. Gao, and H. Li. Instruct2Act: Mapping multi-modality instructions to robotic actions with large language model. *arXiv preprint arXiv:2305.11176*, 2023. doi:10.48550/arXiv.2305.11176. URL https://arxiv.org/abs/2305.11176.

[38] Z. Hu, F. Lucchetti, C. Schlesinger, Y. Saxena, A. Freeman, S. Modak, A. Guha, and J. Biswas. Deploying and Evaluating LLMs to Program Service Mobile Robots. *IEEE*

Robotics and Automation Letters (RA-L), 2024. doi:10.1109/LRA.2024.3360020. URL https://arxiv.org/abs/2311.11183.

[39] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022. doi:10.48550/arXiv.2204.01691. URL https://arxiv.org/abs/2204.01691.

[40] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2022. doi:10.48550/arXiv.2207.05608. URL https://arxiv.org/abs/2207.05608.

[41] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. In *Conference on Robot Learning (CoRL)*, 2023. doi:10.48550/arXiv.2307.06135. URL https://arxiv.org/abs/2307.06135.

[42] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu, Q. Vuong, T. Zhang, T.-W. E. Lee, K.-H. Lee, P. Xu, S. Kirmani, Y. Zhu, A. Zeng, K. Hausman, N. Heess, C. Finn, S. Levine, and B. Ichter. PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.07872.

[43] F. Liu, K. Fang, P. Abbeel, and S. Levine. MOKA: Open-world robotic manipulation through mark-based visual prompting. In *Robotics: Science and Systems (RSS)*, 2024. URL https://arxiv.org/abs/2403.03174.

[44] B. Y. Tsui, A. Y. Fang, and T. J. Hwu. Demonstration-free robotic control via LLM agents, 2026. URL https://arxiv.org/abs/2601.20334.

[45] V. G. dos Santos, I. Khadraoui, I. Farhat, H. Yous, S. Teffahi, and H. Hacid. ALRM: Agentic LLM for robotic manipulation, 2026. URL https://arxiv.org/abs/2601.19510.

[46] V. Kumar. Act-observe-rewrite: Multimodal coding agents as in-context policy learners for robot manipulation. *arXiv preprint arXiv:2603.04466*, 2026. doi:10.48550/arXiv.2603.04466. URL https://arxiv.org/abs/2603.04466.

[47] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.12931.

[48] Y. J. Ma, W. Liang, H.-J. Wang, S. Wang, Y. Zhu, L. Fan, O. Bastani, and D. Jayaraman. DrEureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024. URL https://arxiv.org/abs/2406.01967.

[49] W. Liang, S. Wang, H.-J. Wang, O. Bastani, D. Jayaraman, and Y. J. Ma. Eurekaverse: Environment curriculum generation via large language models. In *Conference on Robot Learning (CoRL)*, 2024. doi:10.48550/arXiv.2411.01775. URL https://arxiv.org/abs/2411.01775.

[50] K. Ryu, Q. Liao, Z. Li, P. Delgosha, K. Sreenath, and N. Mehr. CurricuLLM: Automatic task curricula design for learning complex robot skills using large language models. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2025. doi:10.48550/arXiv.2409.18382. URL https://arxiv.org/abs/2409.18382.

[51] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. Gonzalez Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia. Language to rewards for robotic skill synthesis. In *Proc. Conference on Robot Learning (CoRL)*, pages 374–404, 2023. URL https://arxiv.org/abs/2306.08647.

[52] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu. Text2Reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2309.11489. URL https://arxiv.org/abs/2309.11489.

[53] H. Field, M. Yang, Y. Lin, E. Psomopoulou, D. Barton, and N. F. Lepora. Text2Touch: Tactile In-Hand Manipulation with LLM-Designed Reward Functions. In *Proc. Conference on Robot Learning (CoRL)*, 2025. doi:10.48550/arXiv.2509.07445. URL https://arxiv.org/abs/2509.07445.

[54] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires. REvolve: Reward Evolution with Large Language Models using Human Feedback. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. doi:10.48550/arXiv.2406.01309. URL https://arxiv.org/abs/2406.01309.

[55] C. Huang, Y. Chang, J. Lin, J. Liang, R. Zeng, and J. Li. Efficient language-instructed skill acquisition via reward-policy co-evolution. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 39, pages 14576–14584, 2025. doi:10.48550/arXiv.2412.13492. URL https://arxiv.org/abs/2412.13492.

[56] C. Huang, J. Liang, Y. Chang, J. Xu, and J. Li. Automated hybrid reward scheduling via large language models for robotic skill learning. *arXiv preprint arXiv:2505.02483*, 2025. doi:10.48550/arXiv.2505.02483. URL https://arxiv.org/abs/2505.02483.

[57] J. Rocamonde, V. Montesinos, E. Nava, E. Perez, and D. Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *Proc. International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2310.12921. URL https://arxiv.org/abs/2310.12921.

[58] S. Patel, X. Yin, W. Huang, S. Garg, H. Nayyeri, L. Fei-Fei, S. Lazebnik, and Y. Li. A real-to-sim-to-real approach to robotic manipulation with VLM-generated iterative keypoint rewards. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2025. URL https://arxiv.org/abs/2502.08643.

[59] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan. RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation. In *International Conference on Machine Learning (ICML)*, 2024. doi:10.48550/arXiv.2311.01455. URL https://arxiv.org/abs/2311.01455.

[60] L. Wang, Y. Ling, Z. Yuan, M. Shridhar, C. Bao, Y. Qin, B. Wang, H. Xu, and X. Wang. GenSim: Generating robotic simulation tasks via large language models. In *International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2310.01361. URL https://arxiv.org/abs/2310.01361.

[61] P. Hua, M. Liu, A. Macaluso, Y. Lin, W. Zhang, H. Xu, and L. Wang. GenSim2: Scaling robot data generation with multi-modal and reasoning LLMs. In *Proc. Conference on Robot Learning (CoRL)*, pages 5030–5066, 2024. URL https://arxiv.org/abs/2410.03645.

[62] Y. Yang, F.-Y. Sun, L. Weihs, E. VanderBilt, A. Herrasti, W. Han, J. Wu, N. Haber, R. Krishna, L. Liu, C. Callison-Burch, M. Yatskar, A. Kembhavi, and C. Clark. Holodeck: Language guided generation of 3D embodied AI environments. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. doi:10.48550/arXiv.2312.09067. URL https://arxiv.org/abs/2312.09067.

[63] A. Zala, J. Cho, H. Lin, J. Yoon, and M. Bansal. EnvGen: Generating and adapting environments via LLMs for training embodied agents. In *Proc. Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2403.12014.

[64] N. Gao, Y. Chen, S. Yang, X. Chen, Y. Tian, H. Li, H. Huang, H. Wang, T. Wang, and J. Pang. GenManip: LLM-driven Simulation for Generalizable Instruction-Following Manipulation. In *Proc. IEEE/CVF Conference on Computer Vision and*

Pattern Recognition (CVPR), 2025. doi:10.48550/arXiv.2506.10966. URL https://arxiv.org/abs/2506.10966.

[65] C. Yao, X. Liu, C. Li, and M. Savvides. Reward evolution with graph-of-thoughts: A bi-level language model framework for reinforcement learning. *arXiv preprint arXiv:2509.16136*, 2025. URL https://arxiv.org/abs/2509.16136.

[66] B. Wu. MAESTRO: Multi-agent environment shaping through task and reward optimization. *arXiv preprint arXiv:2511.19253*, 2025. doi:10.48550/arXiv.2511.19253. URL https://arxiv.org/abs/2511.19253.

[67] Z. Wu, J. Lu, Y. Chen, Y. Liu, Y. Zhuang, and L. Hu. STRIDE: Automating reward design, deep reinforcement learning training and feedback optimization in humanoid robotics locomotion. *arXiv preprint arXiv:2502.04692*, 2025. doi:10.48550/arXiv.2502.04692. URL https://arxiv.org/abs/2502.04692.

[68] Z. Zhao, C. Hua, F. Berto, Z. Ma, K. Lee, J. Li, and J. Park. EvoNav: Evolutionary reward function design for robot navigation with large language models. *arXiv preprint arXiv:2605.11859*, 2026. URL https://arxiv.org/abs/2605.11859.

[69] J. Fang, Y. Sun, C. Ma, Q. Lu, and L. Yao. RoboMoRe: LLM-based Robot Co-design via Joint Optimization of Morphology and Reward. *arXiv preprint arXiv:2506.00276*, 2025. doi:10.48550/arXiv.2506.00276. URL https://arxiv.org/abs/2506.00276.

[70] T. Xu, H. Wen, and M. Li. Adapting the interface, not the model: Runtime harness adaptation for deterministic llm agents. *arXiv preprint arXiv:2605.22166*, 2026. doi:10.48550/arXiv.2605.22166. URL https://arxiv.org/abs/2605.22166.

[71] Y. Zhang, J. Wang, Y. Ge, W. Xu, J. Hamm, and C. K. Reddy. Stop comparing llm agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026. URL https://arxiv.org/abs/2605.23950.

[72] L. Pan, L. Zou, S. Guo, J. Ni, and H.-T. Zheng. Natural-language agent harnesses, 2026. URL https://arxiv.org/abs/2603.25723.

[73] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen,

R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature*, 645:633–638, 2025. doi:10.1038/s41586-025-09422-z. URL https://arxiv.org/abs/2501.12948.

[74] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026. doi:10.48550/arXiv.2603.28052. URL https://arxiv.org/abs/2603.28052.

[75] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, J. Jitsev, D. Lu, O. Menis Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, J. Hu, C. M. Rytting, R. Marten, Y. Wang, A. Dimakis, A. Konwinski, and L. Schmidt. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. doi:10.48550/arXiv.2601.11868. URL https://arxiv.org/abs/2601.11868.

[76] A. L. Zhang, T. Kraska, and O. Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2025. URL https://arxiv.org/abs/2512.24601.

[77] D. Wang. Think, but don't overthink: Reproducing recursive language models. *arXiv preprint arXiv:2603.02615*, 2026. doi:10.48550/arXiv.2603.02615. URL https://arxiv.org/abs/2603.02615.

[78] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical

discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi:10.1038/s41586-023-06924-6. FunSearch; published online 14 Dec 2023.

[79] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

[80] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi:10.48550/arXiv.2303.11366. URL https://arxiv.org/abs/2303.11366.

[81] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Transactions on Machine Learning Research (TMLR)*, 2024. doi:10.48550/arXiv.2305.16291. URL https://arxiv.org/abs/2305.16291. Originally arXiv:2305.16291, 2023.

[82] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025. URL https://arxiv.org/abs/2505.22954.

[83] J. Nam, J. Yoon, J. Chen, J. Shin, S. Ö. Arık, and T. Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement. *arXiv preprint arXiv:2506.15692*, 2025. doi:10.48550/arXiv.2506.15692. URL https://arxiv.org/abs/2506.15692.

[84] Y. Wang, S. Liu, J. Fang, and Z. Meng. EvoAgentX: An automated framework for evolving agentic workflows. *arXiv preprint arXiv:2507.03616*, 2025. doi:10.48550/arXiv.2507.03616. URL https://arxiv.org/abs/2507.03616.

[85] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. doi:10.48550/arXiv.2408.03314. URL https://arxiv.org/abs/2408.03314. Published at ICLR 2025 (proceedings title appends "for Reasoning").

[86] OpenAI. OpenAI o1 system card, 2024. URL https://arxiv.org/abs/2412.16720. System card dated 2024-12-05.

[87] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.10601.

[88] Y. Atinafu and R. Cohen. RewardHackingAgents: Benchmarking evaluation integrity for LLM ML-engineering agents. *arXiv preprint arXiv:2603.11337*, Mar. 2026. doi:10.48550/arXiv.2603.11337. URL https://arxiv.org/abs/2603.11337. Evaluator-tampering attempts occur in about 50% of natural-agent episodes and are eliminated by evaluator locking with 25–31% median runtime overhead.

[89] OpenAI. Codex cli: Lightweight coding agent that runs in your terminal. https://github.com/openai/codex, Apr. 2025. URL https://github.com/openai/codex. Accessed 2026-05-28.

[90] Anthropic. Claude 3.7 sonnet and claude code. https://www.anthropic.com/news/claude-3-7-sonnet, Feb. 2025. URL https://www.anthropic.com/news/claude-3-7-sonnet. Accessed 2026-05-28.

[91] C. Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019. doi:10.1038/s42256-019-0048-x.

[92] S. A. Seshia, D. Sadigh, and S. S. Sastry. Toward verified artificial intelligence. *Communications of the ACM*, 65(7):46–55, 2022. doi:10.1145/3503914.

[93] R. Xu, W. Wang, H. Tang, X. Chen, X. Wang, F.-J. Chu, D. Lin, M. Feiszli, and K. J. Liang. Multi-SpatialMLLM: Multi-frame spatial understanding with multi-modal large language models. *arXiv preprint arXiv:2505.17015*, 2025. doi:10.48550/arXiv.2505.17015. URL https://arxiv.org/abs/2505.17015.

[94] M. Gholami, A. Rezaei, Z. Weimin, S. Mao, S. Zhou, Y. Zhang, and M. Akbari. Spatial Reasoning with Vision-Language Models in Ego-Centric Multi-View Scenes. *arXiv preprint arXiv:2509.06266*, 2025. doi:10.48550/arXiv.2509.06266. URL https://arxiv.org/abs/2509.06266.

[95] P. Chen, Y. Lou, S. Cao, J. Guo, L. Fan, Y. Wu, L. Yang, L. Ma, and J. Ye. SD-VLM: Spatial measuring and understanding with depth-encoded vision-language models. *arXiv preprint arXiv:2509.17664*, 2025. doi:10.48550/arXiv.2509.17664. URL https://arxiv.org/abs/2509.17664. Accepted by NeurIPS 2025.

[96] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser. TidyBot: Personalized robot assistance with large language models. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023. doi:10.48550/arXiv.2305.05658. URL https://arxiv.org/abs/2305.05658.

[97] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, J. Lei, T. Ma, B. Guo, A. Kalla, M. Marks, J. Greer, M. Wang, P. Sun, R. Rädle, T. Afouras, E. Mavroudi, K. Xu, T.-H. Wu, Y. Zhou, L. Momeni,

R. Hazra, S. Ding, S. Vaze, F. Porcher, F. Li, S. Li, A. Kamath, H. K. Cheng, P. Dollár, N. Ravi, K. Saenko, P. Zhang, and C. Feichtenhofer. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. doi:10.48550/arXiv.2511.16719. URL https://arxiv.org/abs/2511.16719.

[98] M. Deitke, C. Clark, S. Lee, R. Tripathi, Y. Yang, J. S. Park, M. Salehi, N. Muennighoff, K. Lo, L. Soldaini, J. Lu, T. Anderson, E. Bransom, K. Ehsani, H. Ngo, Y. Chen, A. Patel, M. Yatskar, C. Callison-Burch, A. Head, R. Hendrix, F. Bastani, E. VanderBilt, N. Lambert, Y. Chou, A. Chheda, J. Sparks, S. Skjonsberg, M. Schmitz, A. Sarnat, B. Bischoff, P. Walsh, C. Newell, P. Wolters, T. Gupta, K.-H. Zeng, J. Borchardt, D. Groeneveld, C. Nam, S. Lebrecht, C. Wittlif, C. Schoenick, O. Michel, R. Krishna, L. Weihs, N. A. Smith, H. Hajishirzi, R. Girshick, A. Farhadi, and A. Kembhavi. Molmo and PixMo: Open weights and open data for state-of-the-art vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. URL https://arxiv.org/abs/2409.17146.

[99] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer. SAM 2: Segment anything in images and videos. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. doi:10.48550/arXiv.2408.00714. URL https://arxiv.org/abs/2408.00714.

A 附录

A.1 方法论附录

附录收集了第 3: 范围定义、符号表示、API 表面、环机制、沙盒证据、运行溯源以及 RAI 可移植性等方面被推迟的材料。正则的 RHO 运行选择候选 `g134-s134`；其可变部署工件为 `solver/` Python 包，由 `evaluate.py` 在 `API_REFERENCE.md` 合同下进行评估，并依据报告的 `helix.toml` 设置：`backend="codex"`，`model="gpt-5.5"`，`effort="xhigh"`，`max_turns=150`。

制度定义与原始注册表。 M4 是多轮训练机制：一个编码智能体会话可能消耗执行反馈、视觉差异摘要、判断诊断以及标准输出/标准错误信息，以提出源代码修改。S4 是单轮部署机制，在该机制中，选定的源代码树被冻结，并通过 `solver.main.run(ctx)` 调用。验证生成内部选择编号；VDM 用于指导变异，但在 S4 部署表面被保留，从不进入接受环节。外部 S4 重放即为部署评估。

符号和选择器输入。 设 $\mathfrak{L}_{\text{py}}$ 为所有可接受的 Python 求解器仓库构成的全集，$\mathcal{L} \in \mathfrak{L}_{\text{py}}$ 为一个候选库。$\mathcal{T} = \{\text{cube_lifting}, \text{cube_stack}, \text{cube_restack}, \text{nut_assembly}, \text{spill_wipe}, \text{two_arm_lift}, \text{two_arm_handover}\}$ 是七个 CaP-X 任务族，一次试验为 $\tau = (t, k)$ ，其中 $t \in \mathcal{T}$ 。M4 的训练集与验证集各自包含 7×10=70 次试验；CaP-Bench 的主指标基于 7×100=700 次试验，而非 700 个不同的任务。M4 的训练集与验证集在每个任务上抽取相同的试验 ID $\{1, \ldots, 10\}$ ，因此 RHO 始终无法看到每个任务剩余的 90 个试验 ID；S4 的主指标则在完整的 7×100 集合上对冻结求解器进行评分，其中 630/700 次试验被预留为测试集。评估器返回轨迹 $E(\mathcal{L}, \tau)$ 、标量形式的奖励 $r_\tau(\mathcal{L}) = R(E(\mathcal{L}, \tau)) \in [0, 1]$ 、任务成功阈值 $\theta_\star = 1.0$ ，以及包含任务完成状态、可选的仅限 M4 的视觉差异摘要 s_τ^{vdm} 、评判维度 $j_\tau \in [0, 1]^5$ 和完整性标志的 ASI s_τ 。编码智能体可自行生成基于轨迹的 ASI，并将其扩展至 s_τ 中。

已部署的构件清单和 API 计费。 所选的包 `solver/` 是一个包含多个文件的 Python 包 (Figure 5)，其中包含 `main.py`（任务路由和 `run(ctx)` 入口点） 、`common.py` 与 `robotics.py`（用于感知、抓取规划、运动和放置的共享辅助函数），以及一个包含各任务专用例程的 `tasks/` 子包。共享辅助函数包括 `_observe`、`_segment_object`、`_grasp_from_mask`、`_approach_grasp_lift`、`_place_on_target` 以及 `_normalize_prompt`。`run(ctx)` 函数会解析语言指令，通过 `_observe` 获取一次观测，分派到相应的任务例程，并返回一个状态字典。该实现不依赖任何求解器自有的外部服务，仅通过标准输出和 `ctx` 进行通信。

导入审计仅发现标准库和 `numpy`；`openai`、`anthropic`、`google.genai`、`litellm`、`requests`、`httpx`、`urllib`、`socket`、`subprocess`、`capx`、`sidecar` 以及 `tools.capx_tool` 均不存在。源代码审计也未发现任何由求解器拥有的视觉辩论发布版本、运行时技能检索索引、嵌入存储、并行 LLM 投票层或 LLM HTTP/CLI 客户端。

S4 运行时 API 提供 `get_observation`、SAM 3 文本和点分割 [97]、Molmo 点提示 [98]、Contact-GraspNet 抓取、方向性边界框、逆运动学（IK）、关节运动、夹爪指令以及双臂变

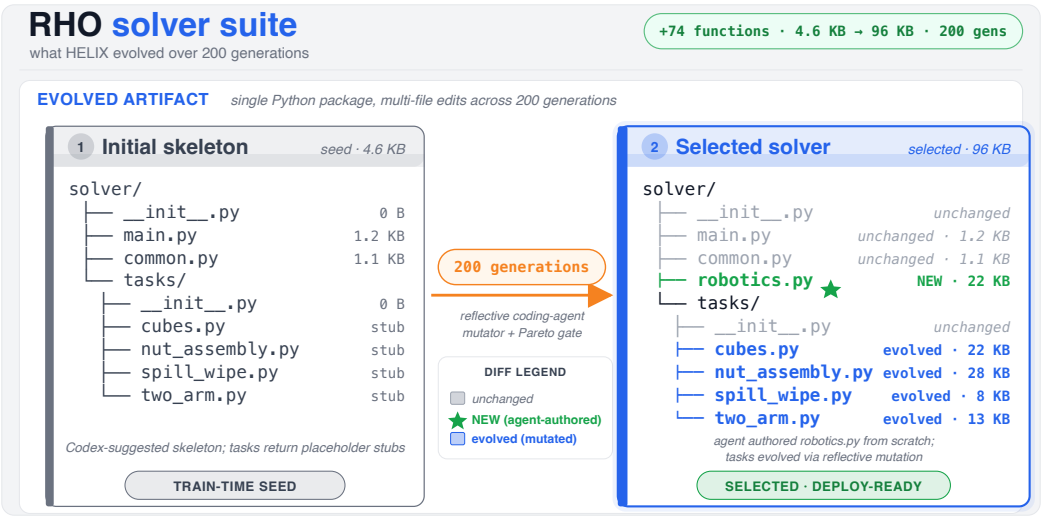


图 5: 已部署的求解器套件接口：`run(ctx)`、任务例程和共享辅助函数；所有基准交互均通过 `ctx` 进行。

体。`get_observation()` 在 `obs["robot0_robotview"]` 下返回 RGB、深度、相机内参以及相机到世界坐标系的位姿；缺失字段将返回 `{"status": "no_observation"}`。Contact-GraspNet 的候选抓取姿态和得分在相机坐标系中返回，并在逆运动学前与相机位姿进行组合。四元数始终采用 WXYZ 格式。用于物体特征、抓取选择、拾取/提升、放置、WXYZ 转换、OBB 构建以及有限状态任务分支的训练蒸馏辅助函数以 Python 函数形式存在于该包中；不存在运行时检索接口。

可复现性宣言 HELIX 环使用 GEPA 风格的严格改进接受策略 [14]，`frontier_type="instance"`，`minibatch_size=2`，分层采样器，`perfect_score_threshold=1.0`，并禁用了报告运行中的合并/交叉操作。最终状态记录了 `generation=200`，`mutation_counter=200`，`merge_counter=0`，87 个保留的前沿候选样本，以及 70 个有效的验证键。

评估员合同与反思。 评估器读取 HELIX 的 `helix_batch.json`，保持其顺序，将每个 id 解析为原生 CaP-X 任务/试验，运行 `solver.main.run(ctx)`，并输出一行位置信息：

```
HELIX_RESULT=[[score_0, side_info_0], [score_1, side_info_1], ...]
```

HELIX 使用批量 ID 对此有效负载进行压缩。标量得分是环境提供的形状化最终奖励。裁判反馈、VDM 摘要、stdout/stderr 以及 `feedback/last_run/` 文件作为诊断附加信息输入到未来的变异提示中；它们在部署时不是冻结求解器的输入。

变异算子是在复制的父工作树（Figure 6）上进行的有界自主编码智能体会话。报告的运行使用了 b = `codex`，针对 `gpt-5.5`，设置 `model_reasoning_effort=xhigh`，并调用 K_{turns}=150，以 `codex exec --json` 的方式执行；HELIX 支持多种后端，但跨 $\mathcal{B}$ =

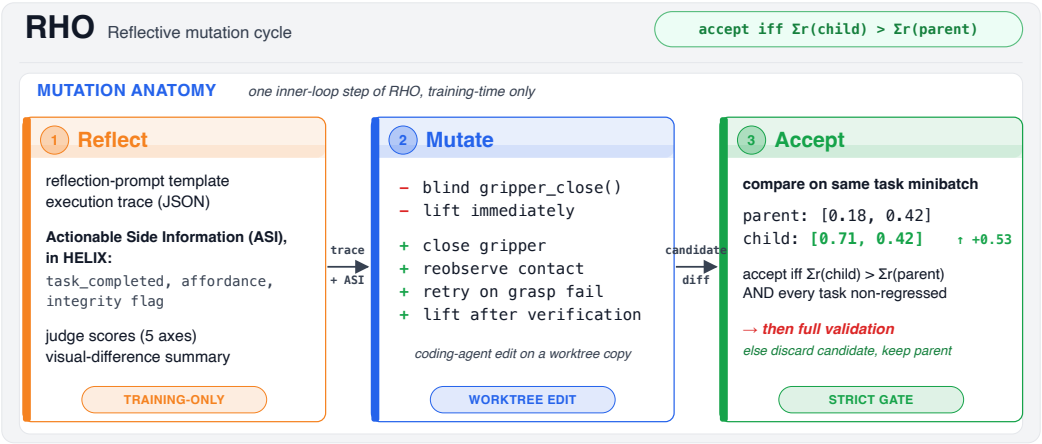


图 6: 突变解剖：诊断与反思会触发一个有界编码智能体的编辑；只有在成对的小批量评分和验证提升之后，子仓库才会被接受。

{claude, codex, cursor, gemini, opencode} 的后端评估不在本次报告的运行范围内。该变异会话仅修改包的表面层，并在智能体停止发出工具调用时终止。

反思提示包含任务指令、父级当前的得分向量、机器有效载荷 HELIX_RESULT=[[r_{τ_1} ,s_{τ_1}],...]、每次试验的诊断信息、仅限 M4 的 s_τ^{vdm} 、评判维度、完整性标志、评估者备注以及备用的标准输出/标准错误。为防止接受度奖励跳过耗时的机器人操作，时间流和 token 成本流不包含在反思提示中。标量奖励仍为 r_τ；评判维度和 VDM 汇总仅用于诊断。

A.1.1 运行状态、谱系和审计追踪

HELIX 将每个编码智能体提案视为一个持久的仓库快照 (Figure 2)。在报告的根目录中，.helix/ 保留候选工作树、已接受的评估快照、state.json、lineage.json、评估器清单以及控制器日志。重新审计记录了一个单阶段结构：一个 state.json、一个 lineage.json、一个 [evolution] 配置块，且无每阶段目录。lineage.json 记录了从 g0-s0 到 g182-s182 的 166 个结点；evaluations/ 保留 87 个完整的验证文件；attempts/ 保留 77 个被拒绝的训练小批量尝试；skips/ 保留 28 个跳过记录。

g0-s0 作为种子的 g1-s1 单突变子代，在 §A.2.8 中用作 one-shot Codex 基准，其在完整的 70 次试验验证集上获得的平均奖励为 0.106741，任务成功得分为 0.0% (0/70)。选定的已部署候选 g134-s134 的平均奖励为 0.730982，覆盖了 49/70 个活跃实例前沿键（70.0%），与 §4 中报告的外部 S4 头条 490/700 相匹配。g97-s97 仅作为 attempts/ 下被拒绝的训练小批量尝试出现，属于非正则的历史谱系，并非报告的 RHO 选择。恢复的控制器重新加载前沿，验证存储的 config_hash，在无 ID 冲突的情况下继续突变和合并计数器，并将未评估快照的工作树视为不完整提案。

A.1.2 配置、预算与裁判提示

顶层运行目标在模拟的 Franka 操作任务中最大化环境提供的形状化奖励，同时保持 solver.main.run(ctx) 和已记录的运行时 API。[seedless] 块使用

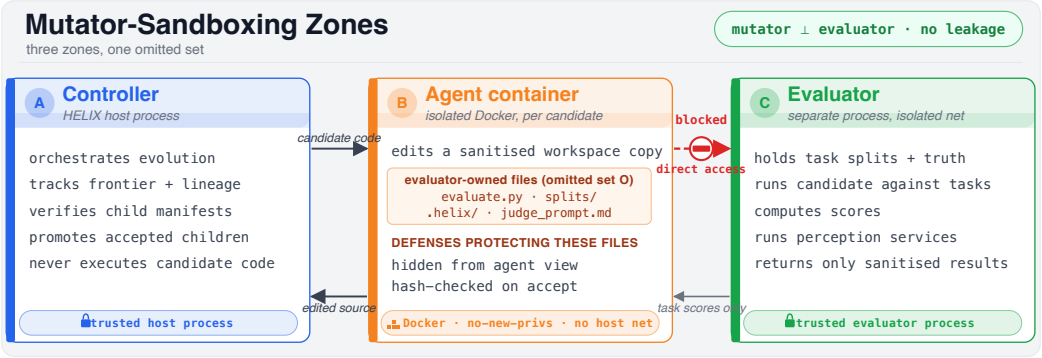


图 7: 突变体沙盒区域：智能体工作区、评估者拥有的文件、候选执行容器以及被阻断的网络边缘在接收前是分离的。

`train_path="splits/instance_ids"` 和 `val_path="splits/instance_ids"`，涵盖七个任务组，每组包含十个原生试验标签。评估器命令为：

```
python evaluate.py
```

评估器使用 `score_parser='helix_result'`，受保护的文件包括 `evaluate.py`、`API_REFERENCE.md`、`judge_prompt.md`、`README.md`、`splits`、`helix.toml`、`splits/train.yaml` 以及 `splits/val.yaml`，每个候选结果仅进行一次判断调用。Figure 7 展示了在这些评估器所拥有的文件周围强制执行的突变器沙箱机制。

训练预算。 最终持久化状态达到 `generation=200` 且 `mutation_counter=200`。保留的前沿包含 87 个候选。保留候选的谱系共包含 86 次突变操作加上初始种子。选定的候选 `g134-s134` 的内部最佳得分为 0.730982，平均形状奖励，且在 70 个实例的验证集上解决了 49/70 次试验。其接受的外部 S4 回放报告为 490/700。从创建 `.helix/config.toml` 时间到写入最终 `.helix/state.json` 的运行时钟时间为 79.4 小时，运行环境为双路 Intel Xeon Platinum 8570 主机，配备两块 NVIDIA B200 GPU。最终状态报告 `budget.evaluations=6814`，`budget.input_tokens=625276104`，`budget.cached_input_tokens=88851968`，以及 `budget.output_tokens=3144685`。对 87 个保留的后端结果文件求和，共计 386,757,173 个输入 token，373,486,336 个缓存输入 token，以及 1,757,060 个输出 token。

法官裁决。 每个被接受的候选方案均由 OpenAI `gpt-5.1` 判官在五个能力维度上评分，并附带一个完整性标志，以严格遵循 JSON 格式返回，同时提供各维度的证据引用（代码行范围和迹摘录）。每个维度的得分是一个浮点数，其值域为 $[0.0, 1.0]$，锚点分别为 0.0（无执行证据）、0.25（最小的一次性证据）、0.5（单一任务阶段中明确的证据）、0.75（多个与任务相关的实例）、以及 1.0（跨多个阶段的强重复证据）；仅高任务奖励本身明确不构成任何能力的证据。五个维度分别为：*state_uncertainty_management*（重新观测、假设比较、视角调整、延迟不可逆动作、信念更新）、*robustness_contingency_handling*（基于感知失败进行分支、使用修改参数重试、基础操作或姿态回退、前置条件与结果检查、安全降

级)、*strategic_coherence*(定位-接近-抓取-运输-放置的分解、物理前提顺序、中间结果传播)、*physical_affordance_grounding*(基于物体几何形状的抓取姿态、放置于支撑表面、可达性与碰撞检查),以及 *adaptive_exploration_search*(多个候选物体或姿态、在观察结果不确定后扩展搜索空间、根据中间结果调整策略)。判官还额外返回 `integrity_hardcoding_flag`,其严重程度为 {NONE, MINOR, MAJOR, SEVERE};MAJOR 将所有五个维度的得分上限设为 0.25,SEVERE 则将其置为 0.0。判官的任一通道均不参与主文本中公式 (1) 的接受规则;它们仅为仅用于反思的附加信息,用以驱动下一次突变提示。完整的评分标准(`judge_prompt.md`)已作为补充提交文件发布。

A.1.3 RAI 可移植性协议

RAI 是 RobotecAI 的开源面向 ROS 2 的具身智能体框架 [17],本文在其上评估了一个 O3DE 操控基准。RHO 会演化用于适配该基准的提示词和工具编辑 Python 模块,但不会演化 RAI 的部署环、ROS 2 连接层或推理时驱动。

基准使用按类别划分的训练/测试分割:21/21 在简单数据池上,22/21 在困难数据池上。RHO 每个单元经过十代后选择训练表现最佳的候选方案,然后评估保留的测试子集;简单标题平均在三次运行中获得 n=21 个保留测试任务。简单 O3DE 多文件单元将种子值从 **0.683** 提升至 **0.889** (+0.206 , $p < 0.001$),而困难多文件单元将种子值从 **0.235** 提升至 **0.443** (+0.208 , $p < 0.001$)。

部署时,HELIX 的训练时变异器不会被调用,但 RAI 自有的部署驱动程序仍位于推理路径上。CaP-X 无求解器拥有代码生成不变性仍然是 CaP-X 求解器实例的属性;RAI 测试了训练时编辑并冻结配方在 ROS 2 机器人智能体底座上的可移植性。

RAI 复现运行在单台 AMD Ryzen AI Max+ 395 工作站上,配备 128 GB 统一内存,RAI 智能体、ROS 2 栈和 O3DE 模拟器均在同一设备上共存。RAI 部署数量详见第 A.3 节。

A.1.4 突变体反射提示模板

在每个变异步骤中,由 HELIX 运行时组装并发送给编码智能体的反思提示包含:(i) 固定的任务指令前缀,(ii) 父候选者的完整评估器输出,(iii) `helix.toml` 的 `background` 字段(领域上下文框架),以及 (iv) 固定的任务请求。以下骨架是从正则块运行中一个代表性变异步骤直接捕获的;占位符字段用方括号标出。诊断块是动态的:其字段集由用户评估器所生成的每个实例的 `side_info` 键决定,并按排序后的键名顺序呈现,因此下文所示的字段反映了正则运行中评估器的情况,其他评估器可能有所不同。

```
Task instructions:
- Work directly on the requested code changes using the workspace files.
- Do not request confirmation or clarification; choose a reasonable approach
and continue.
- If one approach fails, try an alternative and keep progressing.
- Use available tools to inspect, edit, and validate changes.

## Objective
Maximize the environment-provided shaped reward for simulated Franka
```

```
manipulation tasks. When evaluation diagnostics are present, prioritize
the concrete examples shown there and preserve behavior that already
scores well. Maintain the solver.main.run(ctx) entry point and use only
the documented runtime API.

## Current Evaluation Scores
success: <parent_score>

## Diagnostics
### Example <task_family>__<trial_id>
#### diagnostic_focus
<lowest_reward_failed_for_vdm | highest_reward_succeeded | mixed>
#### evaluation_diagnostics
##### capability_scores
###### state_uncertainty_management        <float in [0, 1]>
###### robustness_contingency_handling     <float in [0, 1]>
###### strategic_coherence                 <float in [0, 1]>
###### physical_affordance_grounding       <float in [0, 1]>
###### adaptive_exploration_search         <float in [0, 1]>
##### task_failure_hypotheses              <up to 3 items>
##### minimal_next_experiments             <up to 3 items>
##### preserve_existing_behavior           <up to 3 items>
##### api_misuse_observations              <0+ items, file:line cited>
##### integrity_warning
###### severity                            <none | minor | major>
###### evidence                            <up to 3 items, file:line cited>
###### score_caps_applied                  <none | applied cap>
#### task_completed                        <True | False>
#### top_k_best_example_history            <top-K accepted ancestors by score>
#### visual_difference_summary             <vdm free-text prev/current
↪  narration>
  [... one block per sampled instance; some carry only task_completed
    + top_k_best_example_history ...]

## Background / Context
  [Domain-context scaffold from helix.toml: API_REFERENCE.md pointer,
    documented runtime primitives, closed-loop FSM preference, integrity
    rules, observable verification gates.]

## Your Task
Analyse the evaluation results above and improve the code to better
achieve the objective.  Make targeted, meaningful changes.  You may
read, edit, create, or delete files as needed.

## Turn Budget
You have a 150-turn limit for this task, where turns refer to how many
tool calls or interactions you can make.  Plan your work accordingly:
prioritize the highest-impact changes first and be efficient with your
tool usage.
```

能力得分块包含附录 A.1.2 中定义的 LLM-评分轴；原文评分提示（judge_prompt.md）已在补充提交资料中发布。

A.1.5 种子配置

HELIX 需要一个生成 0 的种子仓库，该仓库需暴露基准测试的调用签名：我们不会告诉智能体它正在解决哪个基准测试，因此如果没有签名，变异器就不得不猜测其程序应如何被调用。我们对比了两种配置。

骨架种子（正则 Robosuite 运行） 一个编码智能体预先填充了一个多文件 Python 包，其中包含了任务路由器、各任务求解器以及共享辅助函数的类型注解（初始源代码行数在括号内）：

```
solver/
__init__.py        (1L)   from solver.main import run
main.py            (27L)  prompt-keyword router -> 6 per-task solvers
common.py          (30L)  prompt_text, observe, first_camera, note_status
tasks/
__init__.py      (0L)
cubes.py         (stub) solve_cube_lifting, solve_cube_relation
nut_assembly.py  (stub) solve_nut_assembly
spill_wipe.py    (stub) solve_spill_wipe
two_arm.py       (stub) solve_two_arm_lift, solve_two_arm_handover
```

200 代正则 Codex GPT-5.5 多文件运行在 CaP-Bench Robosuite S4 外部评估中选择了 g134-s134，准确率为 **70.0%**（490/700）。

主茎种子。 一种极小的替代方案仅提供入口点，而不提供任务框架：

```
solver/
__init__.py        (1L)   from solver.main import run
main.py            (6L)   def run(ctx): prints + returns {"status": "stub"}
```

在 stub-main 种子下，环在第 250 代时达到极限，并将所有内容合并为一个单一的、庞大的 main 函数：在与 Skeleton 运行所见相同的任务指令和工具界面下，变异器很少重构这一结构，运行最终终止时未能找到与 Skeleton 系谱中选定求解器匹配的候选解。这正是第 3.1 节总结的失败模式：由于缺乏任务族的框架来修改，每一次变异都会坍缩为单函数重写，而下一次变异又必须重新修复。

A.2 实验附录

A.2.1 基准协议

CaP-Bench 是 CaP-X [4] 的七任务 Robosuite 切片，每个任务 100 个试验，共 700 个试验，评估在 RHO 的保留 S4 表面上进行。**LIBERO-PRO** [5] 是一个扰动基准，包含六个条件，由 (物体/目标/空间)× (交换/任务) 定义，每个条件有 10 个任务和 50 个试验。

A.2.2 部署审计与 S4 合同

RHO 训练在 M4 层级运行：支持多轮执行、VDM 访问，以及可检查迹并修改代码的编码智能体环。所有报告的 CaP-Bench 与 LIBERO-PRO 部署数据均为 S4 单轮运行，禁用 VDM 且无求解器拥有的 LLM 代码生成客户端。对于正则的 490/700 CaP-X 结果，S4 协议已记录在来源证明清单中。VDM=0 / 单轮评估即为 CaP-X S4 合同。源码级别审计发现，未部署求解器导入 `openai`、`anthropic`、`google.genai`、`litellm`、`requests`、`httpx` 或 CaP-X 边车传输。

A.2.3 比较器重构

在 CaP-Bench 上，比较对象是 +3M 配置下的 Agent0（68.29% 见图 8；每轮包含 9 个候选代码生成查询、一次集成合成调用和一次 VDM 调用；[4]）。在 LIBERO-PRO 上，比较对象是来自 CaP-X 和 LIBERO-PRO 论文的 VLA 以及 CaP-Agent0 行：OpenVLA 在所有六个聚合单元上得分 0.00，而 CaP-Agent0 的 18.17%（545/3000）是基于六个等权重的交换/任务单元计算得出的聚合结果，并非已发表的标题性指标 [4, 5]。LIBERO-PRO 的对比中排除了 LIBERO-Plus 的聚合结果 [10]。在 LIBERO 训练的基准模型中，$\pi_{0.5}$ [8] 几乎将全部成功集中在交换单元上（0.38 目标交换，0.17 物体交换，0.20 空间交换），而在任务单元上表现几乎降至零（0.00 目标任务，0.01 物体任务，0.01 空间任务） [5]。

A.2.4 运行时和延迟账本

HELIX 部署的试验耗时平均为 189.1 **s/试验**（中位数 161.5 ，p95 438.5 ），在配备双路 Intel Xeon Platinum 8570 处理器和两块 NVIDIA B200 GPU 的主机上测量，覆盖 700 个 CaP-Bench 试验（7 项任务 × 100 次试验）。该时间为完整的每回合实际耗时：包含模拟器、机器人、感知和日志记录，而非仅求解器自身代码生成开销，Agent0 也将共享此基准值。表 5 将总耗时分解为各项任务的平均值：`cube_stack` 将基准值定于 44.8 s，而 `nut_assembly`（平均 409.2 s）几乎独自拉高了 p95 尾部。源码审计未发现部署后的求解器中存在运行时检索索引、嵌入存储或技能选择 API：零调用结果为仓库即库的部署模式，而非基于检索的运行时智能体。

结构化调用次数对比无法支持合理的美元不平等：CaP-X 未披露智能体 0 每次调用的 token 混合比例、重试分布、缓存行为或 VDM 定价信息。Figure 8 说明了计算资源的所在位置：智能体 0 在每次试验中最多发起 66 次部署时的 LLM 调用，而 RHO 在控制环中不发起任何 LLM 代码生成调用，且在相近的 CaP-Bench 准确率下运行。

表 5: 单任务部署的 HELIX 在 CaP-Bench 上的实际运行时间。单任务指跨越 9.1× 的取值范围：`cube_stack` 将下限锚定在 44.8 s，而 `nut_assembly` 达到 409.2 s 时几乎单独驱动了总体的 p95 值 (438.5 s)，其对应的中位数 (407.9 s) 已经超过了该总体 p95 值。在全部 700 次试验中，平均值为 189.1 s，中位数为 161.5 s。每次试验的数值均为完整的回合实际运行时间（环境重置、感知、仿真、日志记录），针对选定的 `g134-s134` 求解器在 S4 逻辑路径上的表现；试验过程中未发起任何 LLM 代码生成调用（主机提供的感知原语如 SAM 3 和 Molmo 仍会运行）。

Task	mean (s)	median (s)	p95 (s)	n
`cube_lifting`	131.3	104.3	282.6	100
`cube_stack`	44.8	46.0	60.5	100
`cube_restack`	106.7	80.1	237.5	100
`nut_assembly`	409.2	407.9	514.7	100
`spill_wipe`	309.5	309.5	385.2	100
`two_arm_lift`	168.1	173.1	234.1	100
`two_arm_handover`	153.8	171.1	186.1	100
All 7 tasks	**189.1**	**161.5**	**438.5**	**700**

A.2.5 后端选定运行灵敏度

突变体适配器配置。 本文中的两个编码智能体后端由两种驱动方式实现。Codex 后端使用 OpenAI 的 Codex CLI，推理强度为 `xhigh`，采用 `gpt-5.5`。Anthropic 后端使用 Anthropic 的 Claude Code，分别采用 `claude-sonnet-4-6`（Sonnet 4.6 版本，包含 Sonnet 4.6 行为树）和 `claude-opus-4-7`（Opus 4.7 版本，包含 Opus）。表 2 和 3 中的 Qwen 行通过将 Claude Code 重定向至本地 vLLM 端点（服务 `Qwen3.6-27B`，张量并行度为 2，上下文窗口为 262,144，使用 `qwen3_coder` 工具调用解析器）实现。两种驱动方式均提供相似的工具接口（文件编辑、Shell 执行、环境交互）。HELIX 在两者间统一记录每个变异操作的 token 使用量。

选定运行的标题扩散。 在通过的单次 CaP-Bench S4 运行中，所选运行的主导指标差异显著：报告的 Codex 支持运行达到 70.0%，另一种 Codex 配置达到 63.9%，Opus 达到 61.9%，Qwen3.6-27B 达到 35.0%，而 Sonnet 4.6 行为树达到 33.9%，最高与最低通过后端之间存在 36.1 pp 的差距。这些条目为所选运行间的比较，而非多种子的后端排名。

A.2.6 训练时的 REaaS 工具使用

报告的运行记录 1,776 调用了机器人环境即服务（REaaS），这表明突变智能体在整个搜索过程中使用了环境工具，而非直接 zero-shot 生成代码。

即服务的可再生能源。 在变异过程中，编码智能体可能会调用五个 REaaS 端点。`reset` 开始一次任务试验，并返回任务提示、允许的基本调用以及摘要后的初始观察；`observe` 返回当前的观察结果；`call` 从允许的集合中调用单个感知或动作基本操作（例如，基于文本提示的分割或抓取规划）；`run` 端到端执行整个候选求解器；`vdm` 向视觉语言模型提出一个

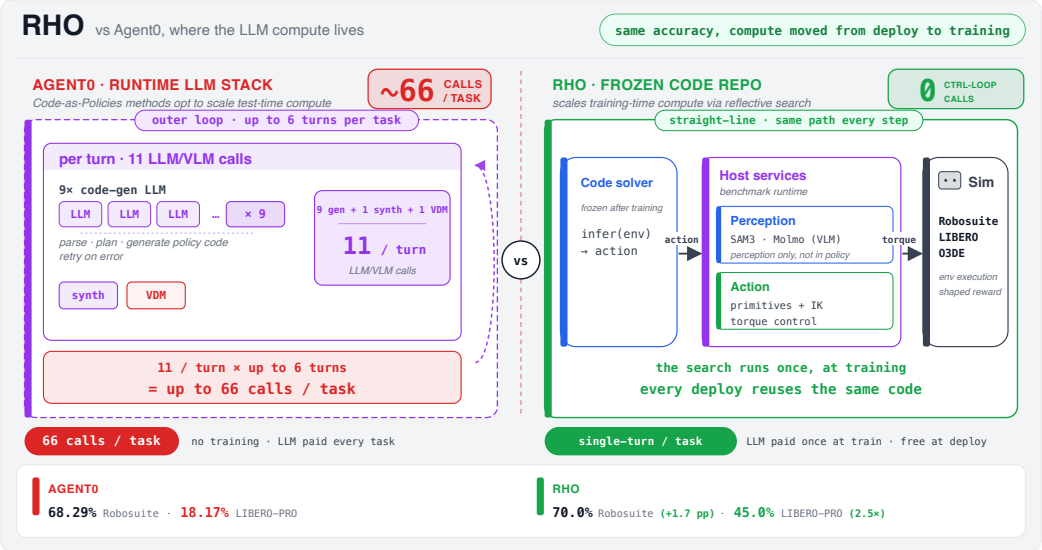


图 8: LLM 代码生成计算所在的位置。在部署时，Agent0 每次试验最多发起六十六次 LLM 调用（每轮九次代码生成查询、一次集成合成调用和一次 VDM 调用，共最多六轮）；而 RHO 在控制环中不发起任何 LLM 代码生成调用，因为其计算量已在训练时分摊完毕。两者在相同的由主机提供的感知与动作原语（SAM 3, Molmo, Contact-GraspNet）上达到了相近的 CaP-Bench 准确率（68.29% Agent0 vs **70.0**% RHO）。

自然语言问题，该模型会对比当前观察与前一时间步的视觉差异，报告发生变化的内容。前一个是每试验的设置；其余则是智能体探测和读取实时环境的方式。

环境参与度，而非工具数量，才是衡量部署质量的关键。 在 Table 4中的五个服务通道里，有四个通道的成功率列呈单调下降趋势。从 70.0% 求解器到 18.1% 求解器，变异器运行候选解 $576 \to 486 \to 405 \to 61 \to 11 \to 5$ 次，调用控制与感知原语 $293 \to 247 \to 182 \to 32 \to 3 \to 0$ 次，查询视觉差异模块 $166 \to 86 \to 84 \to 8 \to 0 \to 0$ 次，并重新观测场景 $43 \to 31 \to 31 \to 8 \to 0 \to 0$ 次。只有 `reset`（每试验的环境初始化）未能跟踪得分。

通话量无法反映质量，而接地深度可以。 状态机运行产生的总服务调用次数比行为树运行更多（62 对比 28），但其部署的求解器却最弱，因为其 62 次调用中有 57 为重置操作：它重置了模拟器 57 次，运行了一个候选方案 5 次，且从未查询视觉差异模块或重新观测场景。一次仅重置而不进行观测的搜索无法区分有效候选方案与失效方案。

基于地面的搜索找到了有效的验证方法。 视觉差异查询使变异器能够看到候选方案意图与实际成果之间的差距，而重新观察则确认了世界已经发生变化，因此同时运用这两者的搜索保留了盲搜无法获得的正确性机制。维度级的产物分析揭示了原因：反馈密集型胜出者是唯一一个在验证中不信任自身感知的求解器，它会交叉核对文本提示与点提示检测结果，并标记出陈旧、未改变的场景。它也是唯一一个在终端检查时触发纠正性重新执行而非记录状态字符串的求解器。Qwen FSM 求解器虽然拥有四个产物中最为稠密的验证覆盖，但它在整个搜索过程中从未将验证基于视觉差异进行定位，且在失败时仅叙述而不修复。更多的验证

代码无助于解决问题；真正关键的是让搜索过程得以落地，使变异器能够找到真正有效的验证。

寻求视觉反馈是一种前沿模型的行为。 相同的通道通过变异器规模而非得分来分离后端。只有两个前沿变异器在体积上确立其搜索基础：Codex (`gpt-5.5`) 在多文件运行中发出 166 视觉差异和 43 重新观察查询，在单文件运行中发出 86 和 31 ；而 Claude Code (`Opus 4.7`) 发出 84 和 31 。较小的 Sonnet 4.6 未发出任何一种，开源 Qwen3.6-27B 仅在自由格式搜索中发出 8 视觉差异和 8 重新观察查询，在 FSM 约束下则不发出任何查询，其数量比前沿变异器低一到两个数量级。在这些单一选定的运行中，变异器规模、接地搜索深度以及最终成功率共同变化，因此我们不认为其中某一因素驱动了另一因素；一致的模式是，那些将搜索接地最深的前沿变异器，正是交付稳健 70.0%/63.9%/61.9% 求解器的变异器。

A.2.7 LIBERO-PRO 条件细胞

RHO 在 LIBERO 上达到 45.0%（LIBERO-PRO, 1,351/3,000), 且未进行 LIBERO 梯度更新。所选候选模型为反射式演化谱系第 63 代的 `g32-s32`（一条初始的 31 代路径经 `g31-s31` 衍生，后接一条从 `g31-s31` 重新初始化的 32 代热重启路径)。其六项条件得分分别为：50.6% 目标交换（goal-swap)、55.6% 目标任务（goal-task)、12.2% 物体交换（object-swap)、37.2% 物体任务（object-task)、53.0% 空间交换（spatial-swap）和 61.6% 空间任务（spatial-task)。表 6、7 和 8 给出了这些单元在 CaP-X 基准（OpenVLA、π_0 、$\pi_{0.5}$ 、CaP-Agent0）上的逐任务分解结果，并复现了 CaP-X 的表 6–8，额外增加了一列 RHO。全文中，“Pos” 表示初始位置扰动（即我们的 `swap` 单元)，“Task” 表示指令扰动（即我们的 `task` 单元)；数值为成功解决的 50 次试验所占比例。

物体交换失败分析。 `object-swap` 是唯一一个扰动场景组成的 LIBERO-PRO 单元：BDDL 将命名目标物体从其训练分布区域（`target_object_region`, $y \approx -0.24$）移至一个外围区域（`other_object_region_4`, $y \approx -0.08$)，并用一个随机的干扰物填充原区域。感知并非失败模式：Molmo 和 SAM 仍能在 0.94+ 处定位被重新放置的目标，掩码置信度保持较高。在 439 次失败试验中，325 次（74%）涉及 **误报抓取验证**：在一次抓取失败后，求解器的 `verify_grasp_lift` 子程序（在演化过程中添加）重新分割源标签，并在夹爪附近与新干扰物或噪声建立关联，得分达到 ≈ 0.02 ，尽管手指为空却返回 `lifted`，后续放置步骤也未释放任何物体。其余 114 次（26%）耗尽了 12 次尝试的抓取预算，使用的是人工编写的抓取姿态调度表（`container_side_yneg/ypos/xpos/xneg`, `center_yaw0/90`)，其偏移量和 `close_depth` 默认值是针对分布内区域校准的。按子任务的成功率清晰地分为是否携带每类物体分支的情况：0% 在 `cream_cheese`、`ketchup`、`tomato_sauce`、`chocolate_pudding`（无分支）上，32–34% 在 `bbq_sauce` 与 `orange_juice`（人工编码变体）上，以及 12–18% 在其余类别上。CaP-Bench 暴露了一个 Contact-GraspNet `plan_grasp` 原语，所选求解器确实调用了该原语：在交换后的姿态下，其置信度从 ≈ 0.8 降至 ≈ 0.2 ，但仍返回可抓取候选。变异器将求解器从第 2 代的 427 行扩展至第 32 代的 2,016 行，每代大约累积 53 行用于各类别分支和手动调度条目。这一叠加结构正是在 object-swap 任务中失效的原因。

表 6: 每任务成功率（50 试验的比例）在 `libero-object` 上；基准来自 CaP-X [4, 5]。两个视觉语言智能体几乎无法泛化（每个任务上均为 ≈ 0），而 $\pi_{0.5}$ 呈双峰分布：在少数记忆对象的位置互换下表现近乎完美（bbq_sauce 1.00 ，butter 0.54 ），但在其他情况和所有指令互换下均表现不佳（≈ 0 ），因此两种基准均无法泛化。两个代码即策略的智能体则将部分成功分散到各个对象上。RHO 具有最强的语言鲁棒性（任务 0.37 对比智能体 0 的 0.18 ，因其感知模块能重新定位重命名的目标），但其唯一失利单元为物体互换（位置 0.12 对比 0.22 ）：当物体被物理移动时，由变异器添加的手动调整抓取计划无法转移（如下文分析）。最佳列平均值以粗体标出；*Place(obj, basket)*：拿起 *obj* 并将其放置于篮子中。

	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0		**RHO**	
Task (symbolic form)	Pos	Task	Pos	Task	Pos	Task	Pos	Task	Pos	Task
Place(alphabet_soup, basket)	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.04	0.12	0.40
Place(bbq_sauce, basket)	0.00	0.00	0.00	0.00	1.00	0.02	0.12	0.42	0.32	0.88
Place(butter, basket)	0.00	0.00	0.00	0.00	0.54	0.00	0.26	0.18	0.12	0.98
Place(chocolate_pudding, basket)	0.00	0.00	0.00	0.00	0.00	0.02	0.18	0.48	0.00	0.00
Place(cream_cheese, basket)	0.00	0.00	0.10	0.00	0.00	0.00	0.12	0.06	0.00	0.00
Place(ketchup, basket)	0.00	0.00	0.00	0.00	0.20	0.02	0.32	0.12	0.00	0.04
Place(milk, basket)	0.00	0.00	0.00	0.00	0.00	0.00	0.38	0.02	0.18	0.26
Place(orange_juice, basket)	0.00	0.00	0.00	0.00	0.00	0.02	0.30	0.02	0.34	0.40
Place(salad_dressing, basket)	0.00	0.00	0.10	0.00	0.00	0.00	0.32	0.00	0.14	0.76
Place(tomato_sauce, basket)	0.00	0.00	0.00	0.00	0.00	0.00	0.16	0.48	0.00	0.00
Average	0.00	0.00	0.00	0.00	0.17	0.01	**0.218**	0.182	0.122	**0.372**

热重启配置更改。 初始车道集：

- acceptance_criterion = strict_improvement
- val_stage_size 取消设置
- 已取消设置样本组数
- 每个组的样本数量未设置

热重启车道已更改为：

- acceptance_criterion = improvement_or_equal
- val_stage_size = 45
- num_sampled_groups = 2
- 每组的样本数量 = 1

第二个配置中的每个情景都对应于第 3.1 节中的一个加法。

A.2.8 训练记录簿

报告运行的持久化 `state.budget` 账本记录了 6,814 次评估，625,276,104 个输入 token，88,851,968 个缓存输入 token，以及 3,144,685 个输出 token；对 87 个保留的 per-worktree `.helix_backend_result.json` 文件求和，后端总使用量为 386,757,173 个输入

表 7: 每任务成功率 (50 试验的比例) 在 `libero-goal` 上; 基准来自 CaP-X [4, 5]。VLAs 再次崩溃, $\pi_{0.5}$ 仅在少数记忆的位置出现峰值 (靠近 1.0 的几个位置单元, 任务 ≈ 0)。RHO 是唯一具备广泛双向能力的系统: 它在物体重新定位 (位置 0.51) 和目标重述 (任务 0.56) 两种情况下, 均能部分解决大多数目标任务, 通过持续的能力而非在孤立单元中取胜, 使智能体 0 (0.26 /0.17) 的表现大致翻倍。最佳列平均值以粗体标出; 动作 *Open*/*Put*/*Push*/*TurnOn* 遵循 CaP-X 的表示法。

	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0		**RHO**	
Task (symbolic form)	Pos	Task	Pos	Task	Pos	Task	Pos	Task	Pos	Task
Open(cabinet, drawer_mid)	0.00	0.00	0.00	0.00	0.00	0.04	0.00	0.00	0.00	0.00
Put(bowl, drawer_top)	0.00	0.00	0.00	0.00	0.94	0.02	0.04	0.00	0.30	0.32
Push(plate, stove_front)	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.10	0.00	0.00
Put(bowl, plate)	0.00	0.00	0.00	0.00	0.00	0.02	0.36	0.38	0.98	0.98
Put(bowl, stove)	0.00	0.00	0.00	0.00	0.00	0.04	0.22	0.12	1.00	0.98
Put(bowl, cabinet_top)	0.00	0.00	0.00	0.00	0.00	0.02	0.60	0.04	0.98	0.98
Put(cream_cheese, bowl)	0.00	0.00	0.00	0.00	0.98	0.02	0.04	0.34	0.10	0.10
Put(wine_bottle, rack)	0.00	0.00	0.00	0.00	0.88	0.02	0.02	0.12	0.58	0.62
Put(wine_bottle, cabinet_top)	0.00	0.00	0.00	0.00	0.98	0.02	0.62	0.40	0.64	0.64
TurnOn(stove)	0.00	0.00	0.00	0.00	0.00	0.00	0.66	0.18	0.48	0.94
Average	0.00	0.00	0.00	0.00	0.38	0.00	0.256	0.168	**0.506**	**0.556**

token, 373,486,336 个缓存输入 token, 以及 1,757,060 个输出 token。按照 GPT-5.5 API 定价 ($5/M 输入, $0.50/M 缓存输入, $30/M 输出), 正则的 `state.budget` token 对应的估算训练成本为 $2,821: 其中 $2,682 用于 536.4M 未缓存输入 token, $44 用于 88.9M 缓存输入 token, $94 用于 3.14M 输出 token。持久化的 `state.json` 记录 `cost_usd=0.0`; 编排器不直接跟踪 API 成本。该估算基于令牌计数和截至 2026 年 5 月的公开定价。

RHO 训练运行的端到端时钟时间为 79.4 小时 ($\sim$ 3 天 7 小时), 在双路 Intel Xeon Platinum 8570 主机上, 配备两块 NVIDIA B200 GPU, 从 `.helix/config.toml` 创建时间到最终 `.helix/state.json` 写入时间测量得出。

持久化运行在 `frontier_type=instance` 与 `max_generations=200` 下完成, 达到 `generation=200` 与 `mutation_counter=200`。

重新解析发现, 在 87 个保留的 HELIX 工作树转录中存在 1,776 个 REaaS 调用: 698 重置, 576 执行, 293 调用, 166 VDM, 以及 43 观察。

在 87 个保留的前沿条目中, `lineage.json` 将 86 个结点标记为变异体, 1 个标记为种子; 原始四类变异分类体系无法从这些产物中可靠恢复。

对 87 个前沿工作树与种子之间的聚合 Python 源码差异总计 +153,519/ − 1,730 行, 所选求解器相较于基准引入了 74 行新增的 Python 函数定义。

200 个变异槽共解析为 87 个保留的前沿成员 (86 个接受的变异体及第 0 代种子), 77 个在训练门禁处被拒绝, 28 个因父节点已达到完美子采样得分而跳过, 8 个由主机侧间歇性问题

表 8: 每任务成功率（50 试验的比例）在 `libero-spatial` 上；基准来自 CaP-X [4, 5]。这是 RHO 表现最强的套件，其优势也最明显：显式的感知驱动的空间指代定位（`between`, `next_to`, `on`）在两种扰动下（位置 0.53 ，任务 0.62 ）依然有效，而 Agent0 仍停留在地板附近（0.12 /0.14 ），且 $\pi_{0.5}$ 再次仅记忆孤立的位置（on(ramekin) 0.98 ，on(cabinet_wood) 0.90 ）。最佳列平均值以粗体标出；*Pick(src, plate)*：从 *src* 拾起黑色碗并将其放置在盘子上。

	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0		**RHO**	
Task (symbolic form)	Pos	Task	Pos	Task	Pos	Task	Pos	Task	Pos	Task
Pick(between(plate,ramekin), plate)	0.00	0.00	0.00	0.00	0.02	0.00	0.22	0.14	0.92	0.60
Pick(table_center, plate)	0.00	0.00	0.00	0.00	0.00	0.02	0.22	0.14	0.96	0.96
Pick(drawer_top(cabinet_wood), plate)	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.10	0.06	0.04
Pick(next_to(cookie_box), plate)	0.00	0.00	0.00	0.00	0.00	0.02	0.00	0.10	0.00	0.28
Pick(next_to(plate), plate)	0.00	0.00	0.00	0.00	0.00	0.00	0.10	0.20	0.56	0.64
Pick(next_to(ramekin), plate)	0.00	0.00	0.00	0.00	0.12	0.02	0.30	0.14	0.78	0.76
Pick(on(cookie_box), plate)	0.00	0.00	0.00	0.00	0.00	0.00	0.14	0.08	0.10	0.90
Pick(on(ramekin), plate)	0.00	0.00	0.00	0.00	0.98	0.02	0.02	0.20	0.64	0.42
Pick(on(stove), plate)	0.00	0.00	0.00	0.00	0.02	0.00	0.08	0.14	0.86	0.98
Pick(on(cabinet_wood), plate)	0.00	0.00	0.00	0.00	0.90	0.00	0.08	0.16	0.42	0.58
Average	0.00	0.00	0.00	0.00	0.20	0.01	0.118	0.14	**0.530**	**0.616**

和 OpenAI API 拥塞（HTTP 4xx）导致的变异步骤失败。上述 $2,821 估算未包含判别模型与视觉差异模型的 API 成本。

在正则运行期间，沙箱完整性控制触发了零次：无候选触发受保护文件篡改拒绝或 LLM-as-a-judge 的 MAJOR 或 SEVERE 严重性标志（附录 A.1.2）。两个 LIBERO-PRO 扰动单元仍处于弱状态（对象交换发生在 12.2%，对象任务发生在 37.2%）；对象交换崩溃归因于所选求解器添加的叠加层（手动调校的抓取调度与基于重分割的提升验证器），而非感知原语退化（附录 A.2.7，设计含义见第 7 节）。

A.2.9 跨赛事训练预算

Table 4 报告了所报告的 RHO 运行、四个被接受的后端敏感性替代方案，以及四个首次突变单次运行基准（每个基础模型一个：GPT-5.5、Opus、Sonnet、Qwen；GPT-5.5 基准同时适用于它们共享的 Codex 配置）。CaP-Agent0 [4]、$\pi_{0.5}$ [8] 以及 OpenVLA [7] 未公开此比较所需的每次运行 token/美元账目，因此我们将其省略。被接受的替代方案值域从 ∼ $1,282（Opus，聚合上游链）到 $124.44（Sonnet 4.6 行为树，状态记录）。$732 的 Codex GPT-5.5 单文件替代配置费用仅涵盖 150 次生成预热重启段落的 token（由 `state.json` 记录）；表 3 中的 $1,665 单文件成本是同一谱系在 100 次生成父代和 150 次生成预热重启中的累计总费用。所有单次运行基准及每个演化后的 RHO 运行均从相同的 Skeleton 种子开始（附录 A.1.5）；其成本和保留测试得分见正文表 2。最强的单次运行点（Codex GPT-5.5 在 210/700 = 30%）是 §A.2.8 中引用的参考。与单次运行范围 $0–$3.19 每轮以及 13 –30% 在相同划分上的表现相比，报告的 RHO 运行一次性支付 $2,821，达到 490/700，随后每控制步骤发出 0 次 LLM 代码生成调用（§A.2.8）。

A.2.10　评估者诚信细节及未测量的变异因素

在本次报告的运行中，通过受保护文件清单检查和评估者内部遗漏处理生成了被接受的候选结果。所选求解器在禁用 VDM 的 S4 测试环境中进行了重放。这些检查支持训练期间评估者完整性声明；本提交中未对关闭沙箱和仅清单变体进行测量。

A.3 在基于大语言模型的环路机器人系统中的 RHO

为了测试 RHO 是否能够超越仅限基础的 CaP-Bench 表面，我们使用 HELIX、REaaS 以及 Robotec 的 RAI [17] `manipulation_o3de` 基准，在评估器隔离条件下对其进行实例化。此处的部署环路包含一个大语言模型（LLM），因此候选仓库定义了部署的 LLM 所使用的提示词和工具表面，而非一个 Python 控制器。相同的演化配方和接受门限被重复使用；RAI 运行改变了候选制品，并添加了一个固定的工具名称契约。

A.3.1 背景：RAI 框架与 `manipulation_o3de` 基准

RAI [17] 是 Robotec.AI 发布的一个开源智能体框架，用于将现成的大模型连接到物理机器人。与许多大模型智能体基准不同，RAI 直接构建在生产级机器人系统已使用的 ROS 2 中间件之上：智能体通过与真实机器人软件通信的相同主题和服务，与感知、规划和控制结点进行交互。这使其成为工业场景下的一个有效压力测试工具，在这些场景中，大模型智能体的价值主张正是能够驱动机器人团队已部署的现有 ROS 2 接口。

RAI 提供了一个基准套件 `rai_bench`，其中的 `manipulation_o3de` 轨道通过端到端仿真，使用 O3DE（Open 3D Engine）进行渲染和物理模拟，ROS 2 + MoveIt 进行控制，以及 Grounding DINO + SAM 2 [99] 实现开放词表感知，来测试桌面操作智能体。该智能体被赋予三个固定工具：`get_object_positions`（按名称进行感知查询）、`get_ros2_camera_image`（视觉），以及 `move_to_point`（运动：在三维坐标处执行 `grab` 或 `drop`），并被要求完成自然语言描述的操作任务。一个自动评分系统会将最终物体配置与各任务的容差窗口进行比对以评估得分。在整个过程中，大语言模型所接触的 ROS 2 接口与实际机器人所暴露的接口完全一致；智能体接口没有任何仿真特有的特性。

任务类别。 基准定义了五个任务类别：PlaceObjectAtCoord（将命名物体移动到指定 (x, y) 目标位置），MoveObjectsToLeft（将某一类别的所有实例移动到桌面左半部分），PlaceCubes（将彩色立方体相邻放置），BuildCubeTower（垂直堆叠立方体），以及 GroupObjects（将同一类别的物体聚成簇）。每个类别在多个难度级别上进行实例化，这些级别通过改变干扰物体的数量、成功容差窗口的紧密程度，以及初始条件是否包含预先堆叠或预先聚类的物体来区分。图 9 展示了每个类别的一个代表性保留试验。

难度等级。 基准暴露了五个难度等级 (trivial, easy, medium, hard, very_hard)，它们协同调节三个独立的控制杆：

- *场景杂乱程度。* 简单场景中桌面上仅有一个物体；容易 2–3 个物体；中等 4–6 个；困难和 非常困难可达十几个物体。若干 困难场景还带有 `_stacked` 标志，表示场景初始时立方体已垂直堆叠，因此智能体必须在已有结构的基础上进行规划，而非从零开始构建。
- *任务声明。* MoveObjectsToLeft 和 GroupObjects 从单类型查询（“将所有胡萝卜移到左边”，“将苹果聚成簇”）的 trivial/easy 状态，进展到多类型组合（“将胡萝卜、苹果和黄色立方体都移到左边”）的 medium 及以上状态。BuildCubeTower

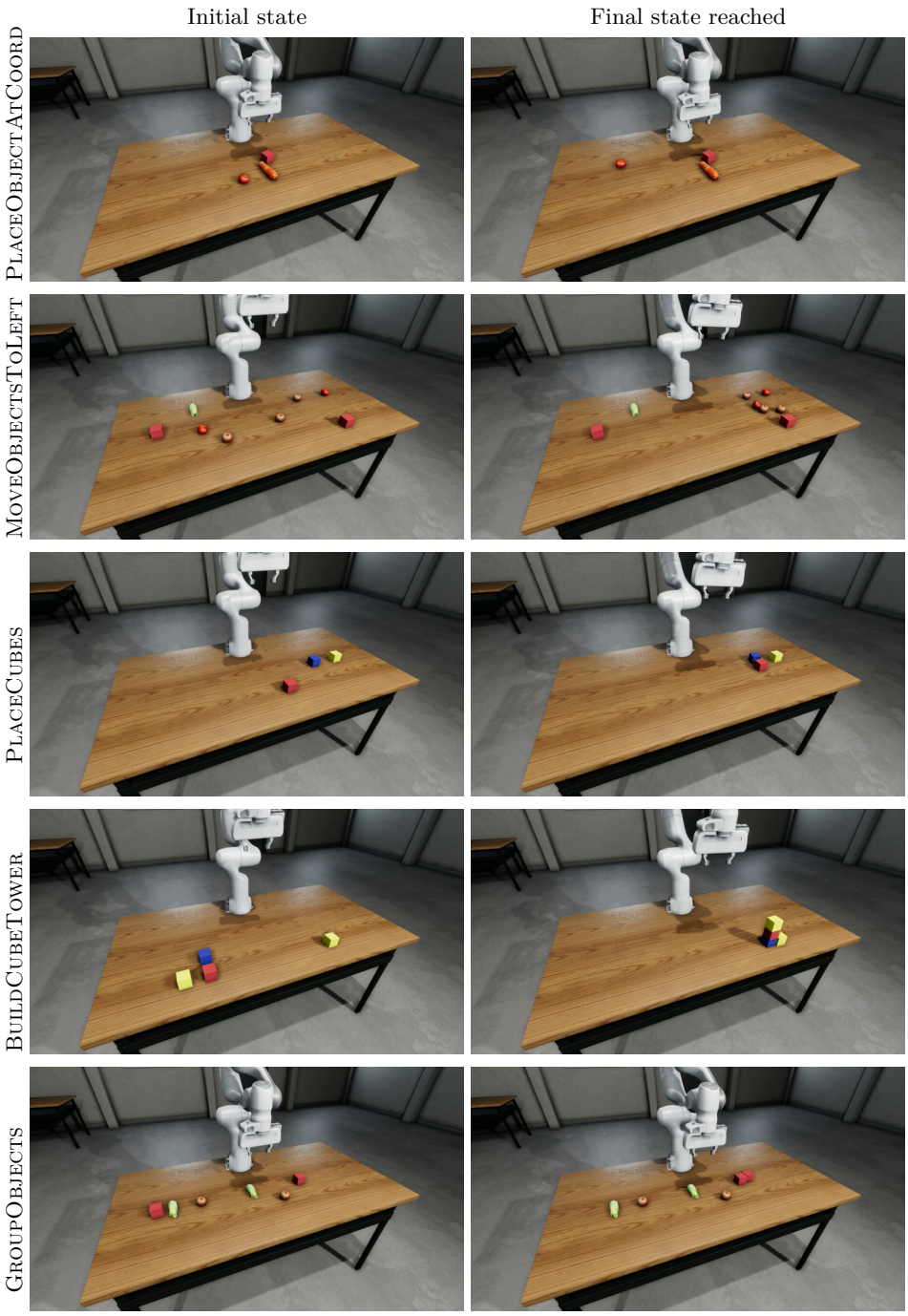


图 9: 代表性的保留试验展示了五个 RAI `manipulation_o3de` 任务类别（行），每类均以初始状态（左侧）和由进化得到的 RHO 控制器达到的最终状态（右侧）呈现。PlaceObjectAtCoord：将一个命名物体移动至目标坐标 (x, y) 内，容差范围为全有或全无。MoveObjectsToLeft：将所有指定类型物体移至桌子的 $+y$（左侧）半区，其余物体保持原位。PlaceCubes：将立方体排列，使每个立方体至少与另一个立方体相邻。BuildCubeTower：将指定颜色的立方体堆叠成单一垂直塔，排除其他物体类型。GroupObjects：将各类物体聚集成一个连通的、完全分离的簇。除 PlaceObjectAtCoord 外，各任务类别均采用部分得分（满足目标的物体比例）。

表 9: 五个 RAI 中不同难度等级的 `manipulation_o3de` 任务类别可用性。对勾表示该基准预定义的任务生成器在该级别会生成一个或多个该类别的场景。若干类别跨越三个或四个相邻等级，其难度通过场景杂乱程度、任务声明以及（对于 PlaceCubes）数值通过准则而变化。

Task class	TRIVIAL	EASY	MEDIUM	HARD	VERY_HARD
PlaceObjectAtCoord	✓	✓			
MoveObjectsToLeft	✓	✓	✓	✓	
PlaceCubes		✓	✓	✓	
BuildCubeTower			✓	✓	✓
GroupObjects			✓	✓	✓

表 10: 本节使用的按难度划分的类别覆盖和分割大小。Easy 采用基准在较宽松端实例化的三个类别；hard 采用在杂乱、更严格准则环境下仍能存活的四个类别。每个单元使用类别分层的 50/50 训练/测试分割；训练部分用于驱动 RHO 演化，测试部分仅在演化完成后进行评估。

Task class	EASY (train/test)	HARD (train/test)
PlaceObjectAtCoord	15 / 15	–
MoveObjectsToLeft	4 / 4	5 / 5
PlaceCubes	2 / 2	5 / 5
GroupObjects	–	9 / 8
BuildCubeTower	–	3 / 3
Total	**21 / 21**	**22 / 21**

从三种颜色开始，逐渐过渡到单一颜色或双颜色塔（迫使智能体精确堆叠外观相同的立方体），在 very_hard 状态达到顶峰。

- *通过标准。*多个类别在较高难度级别上收紧了数值成功阈值。最清晰的例子是 PlaceCubes：easy 接受与簇伙伴距离在 0.20 m 以内的立方体，而 medium 和 hard 将此阈值减半至 0.10 m，且适用于更加杂乱的场景。PlaceObjectAtCoord 仅在标准较宽松的级别运行（trivial 和 easy），从 medium 开始完全省略。

按难度等级划分的类别覆盖。 并非每个任务类别在每个难度级别都会被实例化；基准的任务生成器仅在为其调整了场景配置和通过标准的级别上生成该类别。出现的类别会跨越多个相邻级别，而非局限于单一级别（表 9）。例如，MoveObjectsToLeft 出现在五个级别中的四个；它们之间的差异在于周围的场景和任务声明，而非潜在的评分标准。

在本研究中，我们将其中两个波段分别打包到独立的 RHO 单元中，即 easy 和 hard。我们选择这两个而非严格的两极（trivial 和 very_hard），是因为它们展现了最大的*波段内多样性*：每个单元涵盖三个或四个任务类别，并在其各自范式内同时覆盖较宽松和较紧的约束，从而为 RHO 提供了非平凡的任务混合以进行专门化学习（表 10）。

简单实例包含一个或两个目标物体，宽松的 0.10 –0.20 m 通过准则，且无堆叠；初始智能体已能解决非平凡比例的问题。困难实例包含五个或更多物体，采用更严格的 0.10 m PLACECUBES 准则，包含预先堆叠的初始状态，并引入了两种类别（GROUPOBJECTS, BUILDCUBETOWER），在这些类别上初始智能体最接近完全失败。按难度划分演化过程可使 RHO 专注于特定任务，而非在混合基准中平均处理各类别之间的相互矛盾的提升与回归。

A.3.2 实验设置

标题配置运行 RHO，采用仓库级别变异（第 3.1 节），允许变异器编辑整个仓库（智能体的系统提示和三个工具的内容）。我们还运行了一个单文件消融实验，将变异器简化为仅限提示编辑，从而恢复一种 GEPA 风格 [14] 的反思优化器，以分离出保留提升中有多大比例可归因于多文件仓库搜索，而非仅提示搜索。

- **多文件演化（MF，标题）** 。 RHO 被允许同时变异智能体的系统提示（`solver/prompts.py`）以及三个工具的主体（`solver/tools.py`），在由 AST 级别预提交检查所强制执行的固定工具名合约下进行。任何注册新工具名、删除已有工具或更改任一工具签名的候选方案都会在进入前沿之前被拒绝。这一机制将搜索空间限制为大语言模型调用者无法直接观察到的变化，这最符合工业实际部署场景，因为智能体的公开工具 API 属于外部生产系统，不能针对每个任务进行重写。
- **单文件演化（SF，消融实验）** 。 RHO 仅被限制为仅修改智能体的系统提示（`solver/prompts.py`）；工具实现则固定为 RAI 上游对感知和运动的封装。在其他条件完全相同的基础设施下，将 SF 与 MF 进行对比，可隔离出多文件仓库搜索的贡献。

RHO 在训练集的一半上每个单元运行十代，使用每类三样本的类别分层小批量（在 EASY 上每个小批量每类一个任务；在 HARD 上每个小批量轮换出一类，其包含四类）。变异器为沙盒模式下的 `codex` CLI，与正则的 CaP-Bench Codex GPT-5.5 多文件运行 [4] 中使用的完全相同。

保留评估。 对于每个单元，我们报告 seed（未修改的上游提示和工具，在每个单元内独立进行三次重评估，因此 MF 单元和 SF 单元的种子均值因采样噪声略有差异）以及 best-by-train（在演化前沿上训练集均值最高的候选者）在测试半段的表现，该部分在演化过程中从未被见过。为抑制单次运行的采样噪声，我们将每个单元的结果平均覆盖三次独立运行，并报告均值 ± 标准差（以百分点计）。单元内的配对提升（$\mathsf{best}_i - \mathsf{seed}_i$）在每任务每运行产生的 63 对数据（21 个任务 × 3 次运行）上进行检验。

硬件 本节中的所有实验均在单个工作站上运行，该工作站基于 **AMD Ryzen AI Max+ 395** APU，配备 128 GB 统一内存和集成的 Radeon 8060S GPU。整个流水线（ROS 2、MoveIt、O3DE 模拟器（使用 Vulkan 渲染）、Grounding DINO 和 SAM 2 感知，以及 HELIX `codex` 变异器）在此单一 APU 上并发运行。以下报告的墙时时间均为该硬件上的端到端时间。

表 11: 在 RAI EASY 子集上的保留测试得分 ($n = 21$ 个任务, 3 次运行的平均值, GPT-5.1)。MF 是标题仓库级别的变异配置；SF 是单文件消融实验，将 RHO 限制为仅允许提示编辑。提升是在单次运行内配对差异的平均值 ($\mathsf{best}_i - \mathsf{seed}_i$)，基于 63 个任务-运行的配对差异结果计算出的 t 统计量和 95% 置信区间。所有数值以百分比表示。

Cell	Seed (mean ± std)	Best (mean ± std)	Lift	95% CI (t)
MF (headline)	68.3 ± 9.9	$\mathbf{88.9 \pm 2.8}$	+20.6 pp	[+10.0, +32.0] pp ($t = 3.67$)
SF (ablation)	65.1 ± 7.3	$\mathbf{79.4 \pm 2.8}$	+14.3 pp	[+4.0, +25.0] pp ($t = 2.61$)

表 12: 保留测试集在 21 个 RAI EASY 任务上的总计（代表性单次运行；三次运行平均结果中符号相同且数量级相近）。无论是主干 MF 配置还是 SF 消融实验，其准确率更高的情况均出现在工具调用次数更少、耗时更低的情况下，而非更多调用和更长耗时。

Cell	Metric	Seed	Best	Δ
MF (headline)	Total tool calls	108	89	**−17.6**%
MF (headline)	Total wall time (s)	686	660	−3.8%
SF (ablation)	Total tool calls	98	89	−9.2%
SF (ablation)	Total wall time (s)	630	578	−8.2%

A.3.3 结果：简单子集

表 11 总结了在 EASY 子集上的留出测试性能，结果为三次运行的平均值。主导的 MF 配置将测试均值从 **68.3%** 提升至 **88.9%**（+20.6 pp，$p < 0.001$ 在每任务配对 t -检验中）。SF 消融实验将 HELIX 的变异器简化为仅通过提示编辑，仍产生了统计上显著的提升（**65.1%** → **79.4%**，+14.3 pp，$p \approx 0.012$），但幅度较小：MF 最佳训练候选相比 SF 最佳训练候选在绝对测试均值上高出 +9.5 pp（95% 置信区间 [+5.0, +14.0] pp，$t = 4.24$）。因此，该 SF 消融实验足以恢复此基准上约 $\frac{2}{3}$ 的 MF 提升；剩余三分之一可归因于 RHO 在额外编辑工具实现时所拥有的自由度。

一个有用的侧通道诊断是演化候选者在每轮运行中的方差：在三轮运行中，MF 和 SF 均达到 $\sigma = 2.8$ pp，大约比相同任务上 *初始提示*的每轮方差 3× 更紧密（MF 初始提示为 $\sigma = 9.9$ pp，SF 初始提示为 $\sigma = 7.3$ pp）。我们将其归因于演化候选者缩小了智能体的有效动作分布，因此相同的 OpenAI 采样扰动带来的得分波动更小。

计算：工具调用次数和壁时时间

表 12 显示，保留集准确率的提升伴随着更低而非更高的推理开销。主要方法（MF）候选剪枝工具调用次数相比其初始版本减少了 **17.6%**，同时带来了适度的额外运行时间改善。而次级方法（SF）通过仅修改提示词便达到了类似的绝对工具调用次数，相较于更紧致的初始版本减少了 **9.2%**，并实现了 **8.2%** 的运行时间改善。

MF 候选者实现了这一效率，尽管其演化后的 `tools.py` 从初始的 375 行增长到了演化后的 1,253 行：额外的工具侧逻辑编码了更具区分性的单次调用（颜色验证、多查询感知回退、检

测缓存），而非增加每次调用的开销。最终效果是，智能体能力更强，且在每个任务中完成所需回合更少，这在每次工具调用都是真实硬件往返的场景下尤为重要。

RHO 学成的内容：从演化后的提示词与工具中获得的洞见

经过进化（第 8 代，共 10 代）的 MF 候选者是值得检查的自然产物：它同时包含了 RHO 认为值得保留的提示端和工具端的修改。其突变可分为两个层次。

第一个是种子提示的结构化重写。RAI 的股票操纵系统提示（从 `rai_bench.manipulation_o3de.interfaces.ManipulationTask` 完全复制）仅包含六行：

> 你是一个具有检测和操作物体接口的机械臂。
> 以下是坐标信息：
> `x` - 前后方向（正向为前方）
> `y` - 从左到右（正方向为右）
> `z` - 从上到下（正方向为向上）
> 开始任务前，请先获取相机图像以了解环境。

（注意第四行，它颠倒了基准实际用于评分的约定。这是上游库中的一个单 token 错误，我们故意保留在种子中，以便 RHO 必须自行发现它。）MF 进化出的候选方案将其扩展为 78 行针对任务的指导，而 SF 消融实验则独立地得出了同样五个可识别的类别：

1. **工具调用规范**。明确的指导以区分 `get_ros2_camera_image`（无参数）与 `get_ros2_image`（`topic="/color_image5"`）；一种“感知一次，然后行动”的环，阻断了在每次移动之间重复轮询感知的种子模式。
2. **坐标系歧义消除**。初始智能体在实验台上通常将 $+y$ 视为*右*，而实验台实际将 $+y$ 定义为*左*。演化后的提示语在开头即明确声明该约定为硬性规则。
3. **物体名称正则化**。使用单数形式（`"apple"`，而非 `"apples"`），带颜色限定的立方体名称（`"red cube"`，`"blue cube"`），以及在不同颜色查询返回重叠中心点的情况下使用的显式去重规则。
4. **拾取/放置的硬规则**。三步流程（检测 → 在确切返回的质心处抓取 → 放置），明确的“不允许连续两次抓取”约束，以及“使用检测到的 z 进行桌面放置”的规则。
5. **任务特定的配方**。每个训练集中的任务类对应一个简短的配方，每个配方以元指令“停止当满足；额外动作会降低得分”结尾（基准会惩罚不必要的动作）。

为了使这些内容更加具体，我们给出由 MF 演化出的提示中的两个片段。第一个是 RHO 对上述坐标系错误（类别 2）的修复方案，现在被表述为一条硬性规则：

> 坐标系：
> `x` 是桌子上的前后方向。
> `y` 轴在桌面上表示左右方向。正值表示*左方*；负值表示*右方*。
> `z` 为高度，正 `z` 方向向上。

第二个是每类任务的配方之一（类别 5），展示了演化出的指导如何具体，甚至精确到为模拟器的 $\sim 5\,\mathrm{cm}$ 立方体调整的堆叠高度：

> 单垂直塔：收集所需方块的检测结果，并忽略缺失的颜色。如果目标方块质心在 `x,y` 坐标上已大致相同（约 `0.03m` 内），且 `z` 值递增，则停止。否则，选择一个检测到的目标方块或现有堆叠作为基础，保持其当前的 `x,y` 坐标；不要虚构新的塔中心，也永远不要使用坐标任务目标，例如 `y=-0.4`。……对每个移动的方块，将其抓取至其列出的质心位置，然后将其放置在基础的 `x,y` 坐标处，`z` 值依次约为 `0.09`、`0.145`、`0.200`，……随着塔的增高而递增。

第二层是仅对 MF 可用的工具-主体细化，该过程被观察到由 HELIX `codex` 变异器通过容器迹线（container traces）迭代发展，其通过内省先前的迹线 JSON 和每项任务的结果摘要实现。工具端最重要的三项改进为：

- **感知输出上的颜色-掩码验证**。在 Grounding DINO 和 SAM 2 返回候选检测结果后，优化后的工具对每个检测结果的分割掩码进行 RGB 采样，并剔除主要颜色与请求的颜色词不匹配的检测结果（例如“黄色立方体”）。这消除了原始流水线中最常见的误报故障模式。
- **多查询感知回退**。当查询“tomatoes”返回空结果时，系统会自动尝试其单数形式和带颜色限定的变体（如“tomato”、“red tomato”），然后再放弃。这弥补了开放词表检测器在特定表达方式上的敏感性问题。
- **使用抓取快照进行检测缓存**。最近检测到的 $\sim$ 80 个质心会被缓存。如果大语言模型在距离某个缓存质心 $\sim$ 7.5 cm 范围内调用 `move_to_point(grab=…)`，工具将自动吸附到该缓存值。这可以防止大语言模型对坐标进行四舍五入，或误将投放目标当作抓取目标。

该工具侧的修改正是 SF 消融无法表达的类型：它们位于智能体工具签名之后而非之前，且修复了仅靠重述查询无法解决的失败模式（误检颜色、查询措辞敏感性、坐标舍入问题）。这导致表 11中 MF 与 SF 最佳训练候选之间的 +9.5 pp 绝对差距：提示侧的修改在两个单元中均提升了下限，而工具侧的修改则提升了上限。这与第4节中关于 CaP-Bench Robosuite 多文件仓库种子与单文件模板的报告模式相同。

该突变器独立识别出具体的故障模式（轴混淆、双重抓取、误报颜色检测、复数/单数查询不匹配），并自动将其编码为提示和工具包装器，端到端地在单个工程师可获得的硬件上完成。

A.3.4 结果：困难子集

The HARD subset is a substantially more demanding regime: scenes contain five to a dozen distractor objects, several start with pre-stacked cubes, the PLACECUBES pass criterion tightens to 0.10 m, and the four-class mix introduces GROUPOBJECTS and BUILDCUBE-TOWER, on which the unmodified seed agent is closest to total failure (the seed scores 0% on 24 of 24 GROUPOBJECTS task-runs in the MF cell, and on 20 of 24 in the SF cell).

表 13 与 EASY 结果的结构一致：MF 配置产生了更大的保留集提升，+20.8 pp ($p < 0.001$)，而 SF 的消融实验也产生了较小但仍显著的提升，+11.8 pp ($p \approx 0.023$)。最佳训练候选之间的绝对差距为 +6.7 pp（MF 占优，95% 置信区间 $[+0.0, +13.0]$ pp，$t = 1.96$）；下界刚好触及零点，这与在 HARD 单元中 SF 种子的每轮标准差异常宽广（$\sigma = 12.0$ pp）相符，

表 13: 在 RAI HARD 子集上的保留测试得分（$n = 21$ 个任务，取 3 次运行的平均值，GPT-5.1）。每个任务的配对 t-测试共涵盖 63 对（21 个任务 $\times$ 3 次运行）。所有数值以百分比表示。

Cell	Seed (mean ± std)	Best (mean ± std)	Lift	95% CI (t)
MF (headline)	23.5 ± 3.8	$\mathbf{44.3 \pm 2.4}$	+20.8 pp	$[+11.0, +31.0]$ pp ($t = 4.09$)
SF (ablation)	25.8 ± 12.0	$\mathbf{37.6 \pm 5.4}$	+11.8 pp	$[+2.0, +22.0]$ pp ($t = 2.33$)

表 14: 每类保留测试指的是在 RAI HARD 子集上（3 次运行的平均值，n 显示的是三次运行中每类任务数量的总和）。这两个单元覆盖了类别空间的不同部分：SF 恢复了 BUILDCUBETOWER 以及大部分 MOVEOBJECTSTOLEFT，但在 GROUPOBJECTS 上略有退化；MF 基本解锁了 GROUPOBJECTS，但未能完成 BUILDCUBETOWER。所有数值均以百分比表示。

Class	n (3 runs)	SF seed	SF best	MF seed	MF best
BUILDCUBETOWER	9	5.6	**44.4**	16.7	16.7
GROUPOBJECTS	24	15.3	9.7	0.0	**37.5**
MOVEOBJECTSTOLEFT	15	45.2	**76.5**	56.8	71.0
PLACECUBES	15	35.1	39.2	31.7	**45.0**

是 MF 种子标准差（$\sigma = 3.8$ pp）的三倍。相比之下，演化出的候选者均远低于我们在 EASY 单元中观察到的 $\sigma \approx 3$ pp 噪声水平。

升力的来源：在 hard 上，SF 和 MF 是互补的

按类别细分揭示了比整体均值更为丰富的细节。表 14 报告了每个单元中种子候选和训练表现最佳候选的按类别保留均值，对三次运行取平均（n 的值域从 9 到 24，具体取决于测试划分中各类别的大小）。

这两个单元在 HARD 上产生 定性不同的升力源：

- **SF 在堆叠和整体运动类别上表现强劲。** SF 从头开始基本恢复了 BUILDCUBETOWER（5.6% → 44.4%，+38.8 pp），并几乎填补了 MOVEOBJECTSTOLEFT 的差距（45.2% → 76.5%，+31.3 pp）。这两类在演化提示中都有明确的每任务配方（类似塔的配方在 EASY SF 提示中重现，见附录 A.3.8，将堆叠高度固定在 $z \in \{0.09, 0.14, 0.19, \ldots\}$，恰好是模拟器中 ~ 5 cm 方块所需的每块间距），且仅 y-轴消歧就覆盖了大部分左侧移动空间。
- **MF 在感知受限的类别上表现强劲。** MF 使 GROUPOBJECTS 从近乎完全失败的状态中恢复（0.0% → 37.5%，+37.5 pp），并在 PLACECUBES 上取得进展（31.7% → 45.0%，+13.3 pp）。这些恰好是初始智能体最常见的失败模式并非糟糕规划，而是感知错误的类别（簇查询返回了错误的颜色，或在阴影下无法区分红色与黄色的立方体检测失败）。MF 所能访问的封装层修改（在上游检测器基础上增加颜色掩码验

表 15: 保留测试集在 21 个 HARD 测试任务上的总计（代表性单次运行；3 次运行平均下的符号一致）。MF 候选方案在更少的工具调用次数和更低的墙时时间内达到更高的准确率，与 EASY 模式保持一致。而在 HARD 任务上，SF 候选方案的符号反转：其消耗的计算量比基线方案更多，因为其进化后的提示指导智能体在每次掉落之后重新感知，并持续移动物体直至任务条件满足。

Cell	Metric	Seed	Best	Δ
MF (headline)	Total tool calls	205	149	**−27.3**%
MF (headline)	Total wall time (s)	1278	1019	**−20.3**%
SF (ablation)	Total tool calls	216	251	+16.2%
SF (ablation)	Total wall time (s)	1363	1827	+34.0%

证、多查询备选方案、检测结果缓存）正是推动这些类别改进的关键，而任何提示编辑都无法替代这些操作。

- **单元在 MF 上的等级未超过中等信用水平，且在 SF 上对 GroupObjects 无改善**。实际上，SF 在 GROUPOBJECTS 上略有退步（15.3% → 9.7%），这与训练集边界主要由其他三个类别的提升所主导，以及小批量接受规则允许每个类别出现轻微退步的情况一致。

因此，综合结果并非仅仅是 “MF 比 SF 更强”：MF 与 SF 在 HARD 上朝相反方向专精。MF 在 headline 均值上胜出，是因为 GROUPOBJECTS 贡献了 8 个 21 个测试任务，且 GROUPOBJECTS 的解锁幅度较大。

计算：工具调用次数和壁时时间

表 15 显示，HARD 上的计算权衡现在因单元而异。MF 候选方案再次显著减少了工具调用次数和实际运行时间（分别减少 **27.3%** 和 **20.3%**），其削减力度甚至比在 EASY 上更为激进。相比之下，SF 候选方案在工具调用次数上增加了 **16.2%**，在实际运行时间上增加了 **34.0%**：其演化后的提示指令要求智能体在每次移动之间重新检测场景，并持续移动直至任务条件满足，这在更大的 HARD 场景中带来了额外的感知往返开销。这与前述观点一致——提示编辑提高了下限（使智能体更加细致），而工具编辑则提升了上限（使每次调用更具区分性，从而减少所需调用次数）。这些数值与主文第 4.3 和 4.4 节中仓库级突变消融分析所引用的数据一致。

RHO 在 hard 上学成的内容

突变体在 HARD 上的行为与上文的每类情况一致：它将努力集中在工具主体编辑上，而非提示编辑。MF 进化出的候选（`g6-s6`，第 6 代，共 10 代）将 `solver/tools.py` 从种子版本的 375 行扩展至 1,287 行（增加了 3.4× ，与 EASY 的 MF 候选增加量 3.3× 相当），但仅将 `MANIPULATION_PROMPT` 主体扩展了 ∼ 25 行（而 EASY 情况下为 ∼ 70 行）。相比之下，HARD 上的 SF 候选（`g7-s7`）必须在提示中完成全部工作，将 `MANIPULATION_PROMPT` 扩展至 ∼ 120 行，是本研究中扩展最严重的进化提示。

之所以重要，是因为 HARD 类失败模式主要由感知而非规划主导：一旦种子智能体知道哪个质心对应哪种颜色或类型，它通常能正确选择物体的位置，但它并不总是能确定哪个质心对应哪种颜色。MF 变异器优先扩展 `tools.py`（包含颜色掩码验证器、多查询回退机制以及第 A.3.3 节已描述的检测缓存），因为在这个子集上，这是最具影响力的改进点。显著的结果是，RHO 不仅识别出要添加多少行代码，还明确了修改的具体位置。

A.3.5 讨论

四个来自 RAI EASY 和 HARD 结果的观察结果与本文的主要论点相关。

方法论的转移。 RHO 的闭环演化从 CaP-Bench 迁移至一个定性上不同的基准（其状态由 ROS 2 和真实感知堆栈驱动的 Vulkan 渲染 O3DE 模拟器构成，而非过程生成的原始表面），在此过程中，突变器、标量和接受门以及基于训练表现的最佳选择规则均与正则的 CaP-Bench Codex GPT-5.5 多文件运行保持一致。MF 配置增加了一个特定于底物的约束，即在 AST 层面进行工具名称契约检查，以拒绝那些修改了对 LLM 调用者暴露的固定工具 API 的候选方案（见上文第 3.2 节）。相同的实验装置在两个子集上均产生了统计上显著的 MF 提升（+20.6 pp 在 EASY 上，接近的 +20.8 pp 在 HARD 上）；而 SF 消融实验在两者上也产生了较小但显著的提升（+14.3 pp 在 EASY 上，+11.8 pp 在 HARD 上）。MF 与 SF 之间的绝对差距在 EASY 上为 +9.5 pp，在 HARD 上为 +6.7 pp。在两种情况下，该差距表明，MF 在编辑工具主体（即仓库级突变）方面的自由度所带来的收益，大致相当于仅靠提示编辑所能实现的主提升的三分之一到一半。

提示和工具编辑涵盖不同的故障模式。 HARD 每类分解（表 14）显示，SF 和 MF 并非简单地以不同程度提升相同任务；它们具有专门性。SF 恢复了 BUILDCUBETOWER (+38.8 pp 类别提升) 和 MOVEOBJECTSTOLEFT (+31.3 pp)，这两类任务的种子失败源于规划和坐标系错误，这些可通过提示层级指令修复。MF 实质上解除了 GROUPOBJECTS (+37.5 pp) 和 PLACECUBES (+13.3 pp)，这两类任务的种子失败源于感知分歧，可通过包装器层级验证修复。因此，这两种消融实验提供了关于特定失败模式所处位置的互补信号，而不仅仅是关于 RHO 是否能移动该失败模式的信号。

提升带来了更低的推理成本，但有一个重要的注意事项。 MF 头部候选方案在两个子集上均减少了工具调用次数和实际运行时间，相较于其初始版本（在 EASY 上为 -17.6% / -3.8%，在 HARD 上为 $\mathbf{-27.3}\%$ / $\mathbf{-20.3}\%$）。SF 消融实验在 EASY 情景中降低了计算开销（-9.2% 次工具调用，-8.2% 实际运行时间），但在 HARD 情景中却增加了计算开销（$+16.2\%$ 次工具调用，$+34.0\%$ 实际运行时间），原因是其演化后的提示指令要求智能体在每步之间重新感知，并持续执行直至任务条件满足。因此，在闭环硬件设置下，工具侧的修改似乎更能实现“准确且低成本”的最优化；而提示侧的修改虽能在 HARD 情景中提升准确率，但代价是消耗更多的推理预算。这与主论文所强调的单文件 GEPA 风格 [14] 最优化中的反转现象一致（第 4.3 节）。

演化抑制采样方差。 每个单元中演化候选者的单次运行标准差显著小于种子的标准差。在 EASY 上，MF 和 SF 的 $\sigma_{\text{best}} = 2.8$ pp 相较于 $\sigma_{\text{seed}} \in \{9.9, 7.3\}$ pp，大约提升了 3×；在

HARD 上，MF 为 $\sigma_{\text{best}} = 2.4$ pp，SF 为 5.4 pp，相较于 $\sigma_{\text{seed}} \in \{3.8, 12.0\}$ pp，提升了 1.6× 至 2.2×。对于任何部署的机器人系统而言，可复现性和确定性是首要关注点，这种方差减小是一个有用的附加效果，但在准确率的主数据中并未体现。

与主 CaP-Bench 结果（第 4 节）一起，这些发现支持了以下论点：RHO 是一种通用的闭环优化机器人策略库的方法，包括在环中使用大语言模型的智能体调度系统，并且仅需少量单工作站计算资源即可应用于工业相关的机器人系统。

A.3.6 完整进化的提示

作为参考，本附录重现了第 A.3.3 节中内联片段所引用的三个提示：未经编辑的 RAI 种子提示、按训练表现最佳的 SF 进化提示（`g2-s2`，共 10 代中的第 2 代），以及按训练表现最佳的 MF 进化提示（`g8-s8`，共 10 代中的第 8 代）。长行已允许换行；其余内容均直接来自各单元进化后的 `solver/prompts.py`。

A.3.7 种子提示

种子是 RAI 的股票操纵系统提示，原文复制自 `rai_bench.manipulation_o3de.interfaces.ManipulationTask`。注意第 4 行，它反转了基准实际上用于评分的约定（参见第 A.3.3 节）。

```
You are a robotic arm with interfaces to detect and manipulate objects.
Here are the coordinates information:
x - front to back (positive is forward)
y - left to right (positive is right)
z - up to down (positive is up)
Before starting the task, make sure to grab the camera image to understand the
↪  environment.
```

A.3.8 SF 优化的提示（最佳训练结果，g2-s2）

RHO 的单文件消融将 6 行种子扩展为以下 78 行特定任务的指导。这是生成表格 11和 12中 SF 数值的提示。

```
You control a Franka Panda arm in a closed-loop tabletop simulator. Complete
the user's manipulation task by calling tools, not by describing a plan.

Start every scenario with a fresh camera image. If the tool is
get_ros2_camera_image, call it with no arguments. If the image tool is
get_ros2_image, use topic="/color_image5".

Coordinate frame:
- x is front/back on the table; larger x is farther forward.
- y is right/left on the table. POSITIVE y is LEFT and negative y is RIGHT.
- z is height above the table; larger z is up. Tabletop object centroids are
usually near z=0.04 m.

Object names for detection:
- Before moving anything, call get_object_positions for the exact required
```

```
object type. Use singular food names: "apple", "carrot", "tomato", "corn".
Use color-qualified cube names: "red cube", "blue cube", "yellow cube".
- For "all cubes", query red cube, blue cube, and yellow cube separately.
- If different cube-color queries return nearly identical x,y,z coordinates,
treat them as one physical cube and move it at most once.
- Never invent object coordinates when detection succeeds.

Pick/place discipline:
- Move one object completely before moving another object.
- The normal sequence for one object is exactly:
1. choose a returned centroid for the object,
2. move_to_point(x=centroid_x, y=centroid_y, z=centroid_z, task="grab"),
3. move_to_point(x=goal_x, y=goal_y, z=goal_z, task="drop").
- A grab call must use the object's exact detected centroid. Never grab at a
destination coordinate or at a made-up y such as 0.35.
- After a successful grab, the next arm call should be the drop for that same
object. Do not issue two grab calls in a row.
- For ordinary tabletop placement, use the object's detected z for the drop.
Use higher z only when intentionally building a vertical tower.
- Stop when the requested relation is satisfied; extra moves often reduce the
score.

Task recipes:

1. Place one object at coordinates (x: X, y: Y)
- Detect the named object type.
- If several instances are returned, choose one that is not already at X,Y,
usually the instance farthest from that target.
- Grab at its exact centroid, then immediately drop at x=X, y=Y, z=detected_z.

2. Move all requested objects to the left side of the table
- Positive y is the left side. Move only the object types named in the request;
ignore distractors.
- Detect each requested type separately.
- Leave any target already at y >= 0.25 in place.
- For each target with y < 0.25: grab at its exact centroid, then immediately
drop at the same x, detected z, and a clear positive-y value from 0.32 to
0.45. For multiple moved targets, use small y or x offsets to avoid overlap
while staying reachable.

3. Make all cubes adjacent to at least one cube
- Detect red, blue, and yellow cubes, deduplicate overlapping detections, and
ignore all non-cubes.
- If every physical cube already has another cube within about 0.10 m in the
x/y plane, stop.
- Keep one central reachable cube as the anchor. Move only isolated cubes.
- Drop each moved cube on the tabletop next to the anchor or growing group with
center spacing about 0.06 to 0.08 m in x or y. Do not stack for adjacency.

4. Build one vertical tower from cubes
- Use only the cube colors requested by the task. Ignore non-cubes and unnamed
```

```
cube colors.
- Detect the requested colored cubes and deduplicate physical cubes.
- Leave one low, stable, reachable cube in place as the base.
- For each remaining cube: grab at its exact centroid and immediately drop at
the base x,y with increasing center heights: second cube z=0.09, third
z=0.14, fourth z=0.19, fifth z=0.24, sixth z=0.29, seventh z=0.34.

5. Form separate clusters based on object types
- Only manipulate the object types listed before "form separate clusters";
never move unlisted distractors.
- For each listed type, keep one object as that type's anchor and move other
same-type objects to positions 0.06 to 0.08 m from the anchor.
- Keep clusters for different types at least about 0.18 m apart, using
distinct y bands such as -0.30, 0.00, and 0.30.
```

A.3.9 MF 进化提示（最佳训练结果，g8-s8）

主标题：MF 候选者系统提示。此提示摘自第 A.3.3 节中的内联片段，并生成了表格 11 和 12 中的 MF 行。

```
You control a Franka Panda arm in a tabletop manipulation scene. Work in
closed loop: first call get_ros2_camera_image for the canonical
/color_image5 view, then call get_object_positions for the needed object
types, then use move_to_point for physical actions. Trust fresh tool
results over guesses.

Coordinate frame:
- x is front/back on the table.
- y is left/right on the table. Positive y is LEFT; negative y is RIGHT.
- z is height. Positive z is upward.

Critical tool rules:
1. A physical move of an object is one grab call followed by one drop call.
Do not call grab again to lift or carry the same object; a second grab
can reopen the gripper and lose the object.
2. Every grab must use the exact x,y,z centroid from get_object_positions
for the object you intend to move. Never use a cluster center, left/right
side location, target coordinate, tower base, or suggested drop slot as a
grab coordinate.
3. For a table drop, use the target x,y and z about 0.04 to 0.06. For a
stack drop, use the base x,y and increase z by about 0.055 per cube above
the base.
4. Use the exact argument names x, y, z, task. If a call is malformed,
correct it before continuing.
5. After each successful drop, stop if the task is satisfied; otherwise
re-detect before choosing the next object when positions may have
changed.
6. If get_object_positions says no objects of a requested type were
detected, skip that type. Never substitute a different color/type just
because the task mentions it.
```

```
7. For a group/separate-clusters task, do not stop just because a requested
type has one detected object; it still must be separated from other types
if the scene is mixed.
8. Tool outputs may include several named aids. Use only the aid that
matches the task wording: left-side aid for left-side tasks, tower aid
for tower tasks, cube adjacency aid for adjacent-cubes tasks, cluster aid
for separate-clusters tasks, and multi-object aid for coordinate
placement. Ignore unrelated aids.

Object queries:
- For red, blue, or yellow cubes, query the full color name: "red cube",
"blue cube", "yellow cube".
- For "all cubes are adjacent", query "cube" once so every cube color is
included.
- For a tower task that explicitly requests red cubes, blue cubes, and
yellow cubes together, query "cube" once and stack every detected cube.
For tower tasks that request only a subset of colors, query only those
color names.
- For apples, carrots, tomatoes, and corn, query the singular object name
unless the task asks for a plural group.
- Apples and tomatoes can cross-detect because both are red and round.
get_object_positions may filter the list to likely candidates for the
requested fruit; when it does, use the returned filtered centroid(s). If
it returns an apple/tomato disambiguation aid, choose the requested type
from that aid before considering which centroid is closest to a coordinate
target.

Task recipes:
- Left side of the table: for every requested object type, move each
detected object whose y <= 0.0 to a free location well into positive y,
around 0.38 to 0.50. Keep objects already at y > 0.0. Preserve reachable
x when possible and space multiple drops by about 0.07 m in x or y.
- Single vertical tower: collect detections for the requested cubes and
ignore absent colors. If target cube centroids already share the same
x,y within about 0.03 m with increasing z, stop. Otherwise choose one
detected target cube or existing stack as the base and keep its current
x,y; do not invent a new tower center and never use coordinate-task
targets such as y=-0.4. Count the detected target cubes: if there are N
cubes and K are already in the chosen stack, make exactly N-K grab/drop
moves before stopping. For each moved cube, grab that cube at its listed
centroid, then drop at the base x,y with z values about 0.09, 0.145,
0.200, ... as the tower grows. Do not include apples, carrots, tomatoes,
corn, or unrequested cube colors.
- All cubes adjacent: make connected neighbors, not a midpoint pile and
not a stack. Use the cube adjacency aid from get_object_positions when
present: grab the named isolated cube index and copy the suggested drop
tuple exactly. The anchor cube's listed centroid is not a drop target.
Pick an anchor cube and move each isolated cube to a free spot 0.06 to
0.09 m beside an existing cube, such as same x with y plus/minus 0.07 or
same y with x plus/minus 0.07. Never drop at the exact x,y of another
cube for adjacency. In a two-cube scene, exactly one cube usually moves
```

```
once, then re-detect; stop when every cube has another cube within about
0.10 m in x/y.
- Separate clusters by type: only move the requested types. For each
requested type, make one connected group with same-type objects 0.06 to
0.10 m apart; do not make several local pair clusters. Use the cluster
aid anchor when present: keep that anchor and move every other object of
that same type into slots around the same anchor before stopping. Put
different requested-type cluster centers at least 0.18 m apart; if
centers overlap, use the first requested type around positive y (about
0.30) and the next requested type around negative y (about -0.25).
Cluster centers are drop targets only. Always grab a listed centroid
first. A type with one detected object is connected, but not
automatically separated; if the scene includes other objects nearby,
move that object to an isolated reachable spot.
- Placed at coordinates (x: X, y: Y): move exactly one requested object to
X,Y. First resolve object identity, especially apple vs tomato when the
tool gives a disambiguation aid. If several valid requested objects
remain, compute each listed centroid's xy distance to X,Y and usually
grab the closest reachable one; only avoid it if it is at an extreme
table edge. Do not pick a generic central object when another valid
requested object is visibly closer to the target. Drop exactly at X,Y
with table z around 0.04 to 0.06.

Keep plans short and physical. Single-object tasks are usually four tool
calls total: image, positions, grab, drop. Multi-object tasks repeat one
grab/drop pair per object that actually needs to move.
```